Table of Contents

1. Problem Statement....1

2. Project Objectives....1

3. Literature Survey....2

3.1. Overview of Deafness and Communication Challenges....2

3.2. Sign Language....2

3.2.1. Definition and Applications:....2

3.2.2. Sign Language Categories....3

3.3. Sign Language Technologies....4

3.3.1. Sign Language Capturing....4

3.3.2. Sign Language Recognition System....7

3.4. Artificial Intelligence for Sign Language Recognition....8

3.4.1. Traditional Machine Learning in SLR....8

3.4.2. Deep learning breakthroughs....9

3.4.3. Convolutional Neural Network (CNN) for SLR....9

3.4.4. R-CNN and Faster R-CNN....10

3.4.5. CSLR and ISLR comparison....11

3.5. Object Detection in SLR: The Role of YOLO....14

3.5.1. Introduction....14

3.5.2. YOLO Architecture....14

3.5.3. YOLO Versions and Sizes....20

3.5.4. Training YOLO for SLR....21

3.6. YOLO vs. other CNN based Detectors....21

3.6.1. Architectural differences....22

3.6.2. Speed, accuracy and performance....23

3.6.3. Global vs local context 23
4. Projects Realistic Constraints 24
5. Alternative Designs 30
5.1. SignForDeaf 30
5.2. Gesture-to-Text Mobile Applications 30
5.3. Intelligent Gloves (IG) 31
6. The Design Solution 32
6.1. Hardware Components 32
6.1.1. Processing Unit (Raspberry Pi 5) 32
6.1.2. Input Device (Arducam Camera Module 3) 34
6.1.3. Output Device (128x64 OLED LCD) 35
6.1.4. Additional components 36
6.2. Software Components 37
6.2.1. Operating system 37
6.2.2. Python 38
6.2.3. Used Libraries 39
6.2.4. Datasets 40
6.2.5. CVAT 43
6.2.6. YOLOv8 detection model 44
6.3. YOLO Detection Model Versions Comparison 48
6.4. YOLOv8-M Detection Model Training 50
6.4.1. Training Environment and Hardware Specifications 50
6.4.2. Training Configuration and Hyperparameters 51
6.4.3. Training Flow work 51
6.4.4. Training Results 55

6.5. System integration 60
6.6. Experimentation 64
6.7. Design limitations 65
6.8. Future work 65
6.9. System Schematic Design 66
7. Project Budget 67
8. Project Schedule 68
9. Conclusion 69

List of Figures

Figure 1: Different Sign Languages Alphabet (ASL – ArSL).....3

Figure 2: Sign Language Technologies [9] 4

Figure 3: MYO armband (EMG-Based sensor) [12].....5

Figure 4: Sign Language Recognition System block diagram [8].....7

Figure 5: Neural Networks Architecture 9

Figure 6: Convolutional Neural Network Architecture [27] 10

Figure 7: Visual representation of Intersection Over Union 15

Figure 8: YOLO Object Detection Progress [31].....16

Figure 9: Original YOLO Architecture [38].....17

Figure 10: YOLO Versions and Sizes Chart [43].....21

Figure 11: Intelligent Gloves for The Deaf 31

Figure 12: System Block Diagram 32

Figure 13: Raspberry Pi 5 [49] 33

Figure 14: ArduCam Camera Module v3 34

Figure 15: 128x64 OLED LCD 36

Figure 16: RaspberryPi 5 case with integrated fan.....37

Figure 17: KArSL dataset samples of the three signers for number (2) gesture. 40

Figure 18: SignBridge dataset creation flowchart. 41

Figure 19: (a) and (b) SignBridge Dataset samples of authors for different gestures. 42

Figure 20: Dataset labelling using CVAT 43

Figure 21: YOLOv8 Network Architecture [62].....44

Figure 22: YOLO versions testing environment 49

Figure 23: YOLOv8m detection model training flowchart.....55

Figure 24: 30 epoch model training results 56
Figure 25: 20 epoch model training results 57
Figure 26: 40 epoch model training results 57
Figure 27: Model's normalized confusion matrix 58
Figure 28: Model training batch 0 59
Figure 29: Model validation batch 0 59
Figure 30: SignBridge system 60
Figure 31: SignBridge system workflow 61
Figure 32: Prediction Flowchart 63
Figure 33: System Schematic Design 66

List of Tables

Table 1: Vision-based and sensor-based approaches comparison [8] 6

Table 2: Isolated SLR Literature Work [8] 12

Table 3: Continuous SLR Literature Work [8] 13

Table 4: YOLO, R-CNN, and Faster R-CNN architectural differences 22

Table 5: YOLO, R-CNN, and Faster R-CNN speed, accuracy, and performance comparison [31] 23

Table 6: Camera model specifications [52], [53] 35

Table 7: YOLOv8 backbone layers 45

Table 8: YOLOv8 neck layers 46

Table 9: YOLOv8 software components 47

Table 10: YOLOv8 detection model sizes performance [60] 48

Table 11: YOLO detection model versions accuracies 49

Table 12: YOLO detection model versions performance 49

Table 13: YOLOv8m model accuracies over different devices 64

Table 14: YOLOv8m model performance over different devices 64

Table 15: Project Budget 67

Table 16: Project schedule 68

1. Problem Statement

Language is a fundamental tool for communication and self-expression, enabling individuals worldwide to connect and share ideas. However, for individuals with hearing disabilities, sign language is the primary mode of communication. Unfortunately, sign language is not universally recognized, taught, or used as spoken language, leading to a significant communication gap. Ordinary people often find it difficult to learn and use sign language due to a lack of exposure, resources, or formal training. This gap creates substantial challenges for deaf people in their daily interactions. There is a lack of universal engineering solutions for overcoming this problem, most of the engineering solutions focus on designing apps or individual devices to translate sign language.

2. Project Objectives

The objectives of this project are to:

- Design a physically independent sign-to-text translation unit capable of real-time sign language translation into readable text.
- Develop a computer vision system to recognize and process sign language gestures.
- Implement the designed translation unit and the computer vision system using Raspberry Pi 5.
- Validate the accuracy of the translation system through extensive testing in real-life scenarios and refine the system through collected feedback.

3. Literature Survey

3.1. Overview of Deafness and Communication Challenges

Hearing loss affects over 5% of the global population, or approximately 430 million people, with nearly 80% of those experiencing disabling hearing loss—greater than 35 decibels (dB) in the better ear [1]. This condition significantly hampers communication, creating barriers to education, employment, and daily interactions. Hearing loss ranges from mild to profound and can affect one or both ears, causing difficulties in hearing conversational or loud sounds. While individuals who are hard of hearing often benefit from hearing aids, cochlear implants [2], and assistive devices, deaf individuals with profound hearing loss primarily rely on sign language, a structured visual language [1], [3]. However, limited knowledge and acceptance of sign language among non-deaf populations restricts its practicality [3], highlighting the need for solutions that bridge this communication divide and foster inclusivity in everyday life.

3.2. Sign Language

3.2.1. ***Definition and Applications:***

Sign language is a fully developed visual-gestural language that enables communication through hand gestures, facial expressions, body movements, and spatial relationships. Unlike gestures used alongside spoken language, sign languages have their own grammar, syntax, and vocabulary, unique to the linguistic and cultural communities they serve [4]. These languages typically emerge among deaf communities as an independent means of communication. However, some hearing groups have also developed their own sign systems, such as those used by Australian Aboriginal women in mourning or Plains Indian signing [5]. Each sign language operates independently, with no direct one-to-one translation to spoken languages.

A common misconception is that sign language is universal, but like spoken languages, they differ widely due to geographical and social boundaries [4]. Additionally, some sign languages, such as American Sign Language (ASL), have influenced others through cultural exchanges and education. Another myth is that sign languages are mere manual representations of spoken languages. In reality, they have distinct grammatical structures

and rules, often developing independently of the spoken languages in their regions [5]. There are more than 300 sign languages in the world, including ASL, BSL, ISL, ArSL, and others, according to the classification system [6]. The following figures illustrate the basic differences between American Sign Language and Arabic Sign Language.

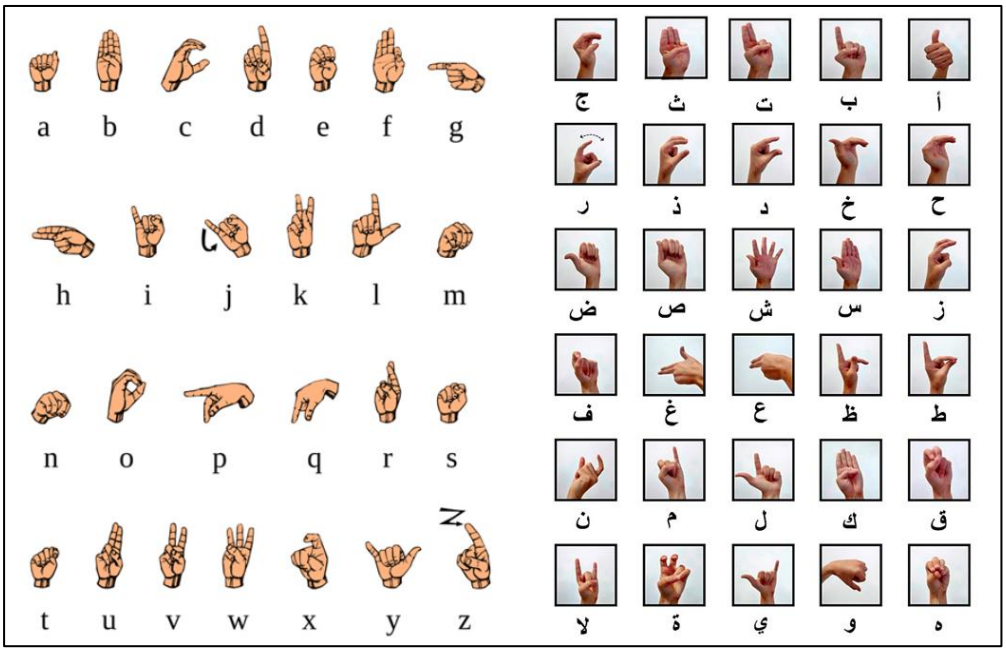


Figure 1: Different Sign Languages Alphabet (ASL – ArSL)

*3.2.2. **Sign Language Categories***

I. Manual and non-manual features

- **Manual Features**: These involve hand movements and configurations, such as the shape of the hands and the motion trajectory [7], [8]. Manual features are the most visible and often the primary means of conveying meaning in sign language.

- **Non-Manual Features**: These include facial expressions, gaze direction, lip patterns, and body posture. Non-manual markers play a vital role in conveying additional meaning, such as indicating negation, differentiating questions from statements, or adding details like size or time [7], [8]. They can complement or alter the meaning of manual signs, making them essential for accurate communication.

II. Static and Dynamic gestures

- **Static gestures**: These gestures involve hand shapes and orientations that remain still, with no motion involved. The meaning of these signs is conveyed solely through the position of the hands, fingers, and other hand characteristics [7].

- **Dynamic gestures**: These gestures include motion along with hand shape and orientation. The movement of the hands or arms is essential to the meaning, as factors like direction, speed, and path of motion can alter the sign's interpretation [7].

3.3. Sign Language Technologies

The development in computer science and engineering has improved the applications in sign language technologies, including sign capturing and data collection to accurate translation and representation to help hearing-impaired people communicate with each other and with hearing people. As shown in Figure 2, Sign language technologies are interconnected and have an impact on one another [9].

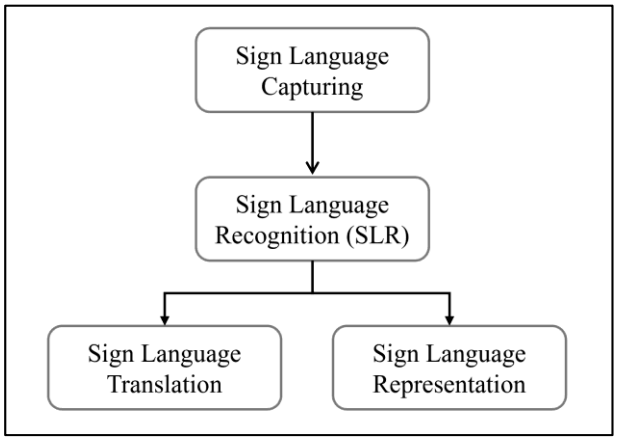


Figure 2: Sign Language Technologies [9]

*3.3.1. **Sign Language Capturing***

Recording sign gestures with the right sensor configurations is known as sign language capturing. The goal is to record discriminative information from the signs so that they can be later studied, recognized, and translated. Additionally, recording sign language makes it possible to compile sizable datasets that can be used to effectively train and assess machine learning algorithms for sign language recognition and translation [9]. Mainly two

approaches are used to capture sign language and collect data, the Sensor-based approach and the vision-based approach.

I. Sensor-based approach

Input is taken as a signal from the sensor in this approach [10]. Various sensor technologies have been employed for capturing sign language gestures, each with distinct advantages and limitations [9]. Kinect sensors are effective for recording RGB and depth data but are limited by distance constraints. Leap Motion offers real-time applications with low computational demands but requires proximity. Wearable muscle movements and motion detector devices capture Electromyography (EMG) and inertial signals but face challenges with movement distortions [11]. Data gloves provide high accuracy but are often considered invasive and require complex calibration [9]. These technologies balance precision, usability, and practicality depending on specific application needs. The following figure shows an example of an EMG-based sensor.



Figure 3: MYO armband (EMG-Based sensor) [12]

II. Vision-based approach

In this approach, the input is an image or video [10]. Numerous types of cameras are used as input units, to collect typical signs one camera is used, and more than one camera (depth camera) detects depth specifications [13], and for uninterrupted motion, a video camera can be used [8]. Due to its viability for real-time applications, vision-based SLR is the topic of a lot of research.

The following table shows the core differences between sensor and vision-based approaches:

Table 1: Vision-based and sensor-based approaches comparison [8]

Method	Vision-based approach	Sensor-based approach
Capturing device	Cameras – Video camera, Depth camera, RGB camera	Sensors and gloves
Obstacle	Environment, disturbance, and noise.	Environment, disturbance, and noise.
Efficiency	Low (depends on the resolution).	Better than vision-based method (depends on sensor performance).
Cost	Low	High
Limitation	Sensitive to lighting conditions, occlusions, and complex backgrounds. High computational cost for real-time processing.	Bulky hardware (e.g., gloves), limited mobility, requires calibration, and may suffer from drift over time.
Advantage	Non-intrusive, scalable, works well for gesture recognition in controlled environments.	High accuracy, robust to lighting changes, works in low visibility.

III. Dataset Types

High-quality datasets are essential for advancing sign language recognition, translation, and synthesis techniques [13]. Consequently, significant focus has been placed on capturing signs accurately and ensuring precise annotation. Most publicly available datasets are created using visual sensors and are highlighted below for their contributions to the field.

1) Continuous Sign Language Recognition Datasets

Continuous sign language recognition (CSLR) datasets are crucial for developing real-life applications, as they contain videos of sign sequences performed continually without pause, ensuring that the context between signs is understood clearly [14].

2) Isolated sign language recognition Dataset

Isolated sign language recognition (ISLR) datasets are essential for learning vital features of sign language recognition, with a focus on individual signs, recorded and

analyzed separately without context, making them suitable for recognizing isolated gestures rather than continuous signing [14].

3.3.2. Sign Language Recognition System

The process of identifying sign language symbols is known as sign language recognition (SLR) [9]. Most frequently an SLR System – as shown in figure 4 – goes through stages of data acquisition, preprocessing, segmentation, tracking, feature extraction, database, classification, and recognition accuracy and error rate computing. In sensor-based it is optional to do preprocessing and segmentation [7].

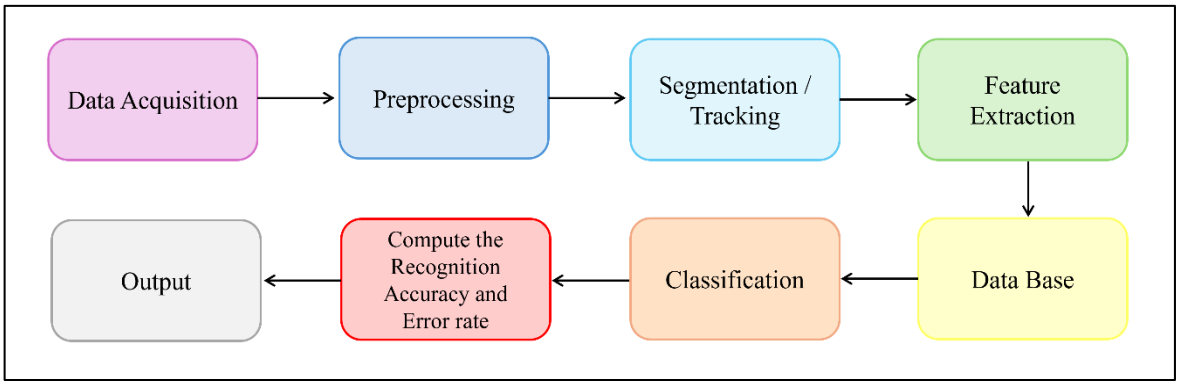


Figure 4: Sign Language Recognition System block diagram [8].

- **Data Acquisition:** One of the two modes – vision or sensor – is used to acquire data in SLR [13]. Core differences between these approaches are explained in section 3.3.1.

- **Preprocessing:** By using preprocessing techniques including dimension reduction, normalization, and noise removal, the SLR system's performance can be enhanced [7].

- **Segmentation:** The segmentation stage divides the image into different regions or areas of interest (ROI). This process may include techniques like Skin Color Segmentation, Hands Tracking and Segmentation (HTS), Entropy Analysis, and the use of the Picture Information Measure (PIM) [15], [16], [17].

- **Tracking:** Effective hand gesture extraction from the background demands a segmentation and tracking procedure [8]. It is challenging because of the quick change in hand position in the frame according to the hand's movement [7].

- **Feature extraction:** The process of converting relevant portions of input data into sets of compacted feature vectors is known as feature extraction [18] Techniques like Shift-invariant feature transform (SIFT) [19], Speeded up robust feature (SURF), Principal component analysis (PCA) [18], and Linear discriminant analysis (LDA) [20], are used for feature extraction.

- **Database:** The acquired data is set aside in the database and categorized as Training dataset – used for classification, and Testing dataset – used for accuracy.

- **Classification:** Refers to the process of categorizing input data (such as hand gestures, body movements, or facial expressions) into predefined classes or labels (e.g., letters, words, or phrases in sign language). Both supervised and unsupervised machine learning approaches can be used for classification [7].

- **Recognition accuracy and Error rate:** The trained classifier performance is verified using the testing dataset – unused dataset while training [8].

3.4. Artificial Intelligence for Sign Language Recognition

3.4.1. Traditional Machine Learning in SLR

Traditional machine learning approaches such as Support Vector Machines (SVMs) and Hidden Markov Models (HMMs) have been historically used for isolated sign recognition [21]. SVMs classify hand shapes using statistical separation, while HMMs model temporal sign sequences through state transitions. However, these methods face limitations in handling the complex spatial-temporal dependencies of continuous sign language [22], requiring extensive feature engineering and struggling with variability in signing styles and environmental conditions.

3.4.2. *Deep learning breakthroughs*

Deep learning has revolutionized SLR by automatically learning hierarchical features from raw data. Convolutional Neural Networks (CNNs) excel at spatial feature extraction from hand postures [23], while Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks model temporal sign sequences. Transformer architecture has further improved performance through self-attention mechanisms that capture long-range dependencies in sign language videos, outperforming classical methods in accuracy and robustness [24].

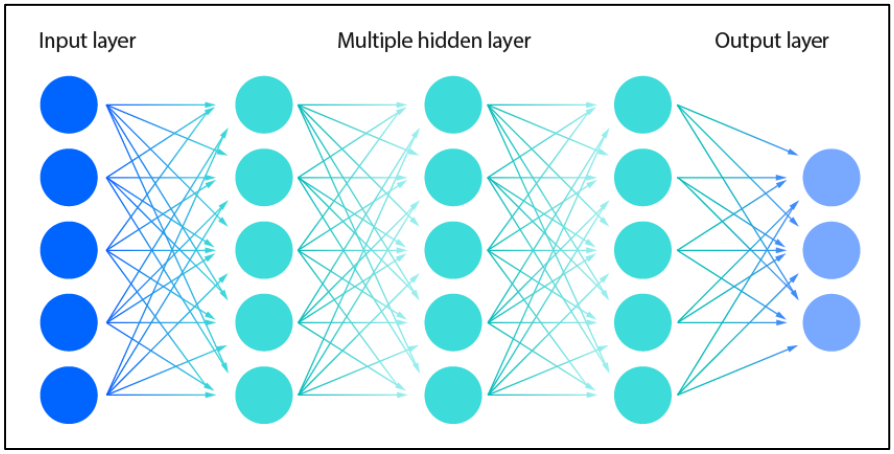


Figure 5: Neural Networks Architecture

3.4.3. *Convolutional Neural Network (CNN) for SLR*

A Convolutional Neural Network (CNN) is a type of artificial neural network designed for processing grid-like data, such as images. CNNs process sign language input through layered convolution and pooling operations to extract increasingly abstract spatial features [25]. Early layers detect edges and basic shapes, while deeper layers identify complex hand configurations and facial expressions critical for sign interpretation. Architectures like ResNet and EfficientNet have been adapted for SLR, demonstrating strong performance in classifying static signs from image data [26].

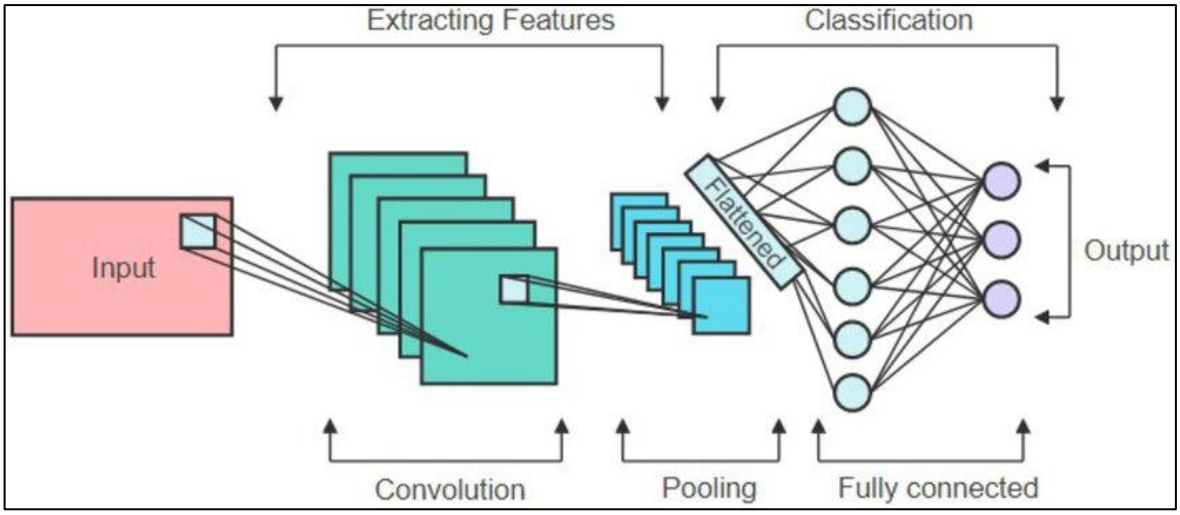


Figure 6: Convolutional Neural Network Architecture [27]

*3.4.4. **R-CNN and Faster R-CNN***

Region-based Convolutional Neural Networks (RCNN) represent a pioneering approach in object detection that combines regional proposals with CNN features. The architecture operates in three key stages [28]:

1. **Region Proposal**: Extracts ~2000 candidate object regions using selective search (traditional RCNN) or Region Proposal Networks (Faster RCNN)

2. **Feature Extraction**: Processes each region through a CNN (e.g., AlexNet/VGG) to generate fixed-length feature vectors

3. **Classification**: Uses SVM classifiers (RCNN) or fully connected layers (Faster RCNN) to predict object classes

Region-based CNNs revolutionized spatial feature extraction in SLR systems. The original RCNN [28] pioneered region proposal methods, achieving 85.7% accuracy on isolated signs by processing ~2000 regions per image. Faster RCNN [29] improved efficiency through Region Proposal Networks (RPNs), reducing processing time by 250x while maintaining 89.2% mAP on dynamic gestures. These architectures excel in handling occluded hands [30] but face challenges in real-time deployment due to computational overhead (average 5 FPS vs YOLO's 45 FPS) [31].

*3.4.5. **CSLR and ISLR comparison***

Both sign language recognition approaches – sensor-based and vision-based – can be classified into Continuous sign language recognition and Isolated sign language recognition [32]:

I. Continuous Sign Language Recognition (CSLR)

Continuous Sign Language Recognition (CSLR) aims to classify signed videos into entire sentences by recognizing ordered sequences of glosses. This task is particularly challenging due to the absence of clear sign boundaries in video sequences, requiring advanced techniques to align input videos with target glosses [14]. Most CSLR methods rely on 2D or 3D Convolutional Neural Networks (CNNs) for feature extraction and use temporal modeling methods such as Temporal Convolutional Networks (TCNs) or Recurrent Neural Networks (RNNs) to process sequential information [9]. Word Error Rate (WER), is used to measure CSLR's performance which evaluates how well the predicted gloss sequence matches the target, considering substitutions, deletions, and insertions [33].

II. Isolated Sign Language Recognition

Isolated Sign Language Recognition (ISLR) demonstrates the transformative power of AI by harnessing advanced techniques like attention mechanisms, multi-stream architectures, and Graph Convolutional Networks (GCNs) to process complex video data with remarkable accuracy [34]. ISLR excels in extracting and learning from discriminative features, such as hand, mouth, and skeletal data, while leveraging multi-modal inputs like RGB, depth, and motion to tackle challenging tasks. Innovations like spatial 3D relational features, color-coded descriptors, and domain-invariant knowledge transfer highlight AI's ability to adapt and generalize across datasets [9]. Attention mechanisms enable precise focus on critical features, enhancing the performance of multi-stream and spatio-temporal models. Furthermore, AI's integration with RGB-D sensors has expanded ISLR's potential, addressing the nuances of gesture recognition. Despite challenges in user variability, ambiguous signs, and limited datasets, AI-driven advancements promise to refine ISLR's robustness, accuracy, and scalability, cementing its role in bridging communication gaps.

The following tables show a comprehensive comparison between tested recognition systems:

Table 2: Isolated SLR Literature Work [8]

Author/Year	Method	Accuracy	Sensor	Model Type	Dataset
Lee et al 2021	KNN, LSTM, and Recurrent Neural Network.	Recognition: 99.44% Validation: 91,8%	Leap motion controller	American SLR	2600 samples
Aly and Aly 2020	Convolutional SOM, deep Bi directional, LSTM network, and DeepLabv3+	Without segmentation: 69.0% With segmentation: 89.5%.	Video camera: 25fps.	Arabic sign language.	3450 samples.
Rastgoo et al. 2020	Midpoint algorithm, SSD, 2DCNN, 3DCNN, and LSTM	96.2% for the RKS-PERSIANSIGN dataset	Video camera (RGB).	Persian signs language.	RKS-PERSIANSIGN (100 signs, 10,000 samples) datasets.
Lee and Lee 2018	Support Vector Machines.	Without pressure: 65.7% With pressure: 98.2%.	Custom glove (5 flex sensors, 2 pressure sensors, and 2 axis inertial motion sensor)	American sign language	6,480,000 datasets.
Wei et al. 2016	Code matching method, and fuzzy K-mean algorithm.	Third of the gesture set: (82.6 ±13.2)% Half the gesture set: (88±13.7)%	sEMG, GYRO, and ACC	Chinese sign language.	13750 samples (2750sign word samples for each subject).

Author/Year	Pros	Cons
Lee et al 2021	Suitable for real-time environment/applications.	One hand sign (right hand) only considered for recognition.
Aly and Aly 2020	Signers independent combinational sign recognition model.	Experimentation performed with a limited number of signers.
Rastgoo et al. 2020	Large-scale hand sign language dataset presented	Complex model.
Lee and Lee 2018	Custom-made devices not required because 3D-printed based device used it can fit different sign irrespective of hand and finger sizes holders.	The crucial factors like background light and other effects fail to be addressed. Sign respect to words and sentences not considered for experiments, and implementation can be on a smaller-sized printed circuit board if the research gap is not addressed.
Wei et al. 2016	User's training burden minimized.	Target set size is limited; instead of code matching method, advanced fusion method can improve the recognition accuracy.

Table 3: Continuous SLR Literature Work [8]

Author/Year	Method	Accuracy	Sensor	Model Type	Dataset
Gurbuz et al. 2021	PCA, short term Fourier transform, and RBF associated SVM.	20 sign, 150 features: 72.5% 5 sign: 95%	RF sensors and Kinect sensor.	American SLR.	240 samples.
Hassan et al. 2019	Modified KNN and HMM.	Sentence recognition, MKNN based model: 97.78% Words recognition, RASR based model: 99.20%	Camera, DG5-V Hand data glove and Polhemus G4 motion tracker.	Arabic sign language.	800 words samples, 400 sentence samples.
Tripathi and Nandi 2015	OH, PCA, and distance-based classifier.	Average recognition rate for Euclidean distance (18 and 36 bins OH respectively): 90% and 91.3% Average recognition rate for correlation (18 and 36 bins OH respectively): 89% and 89.8%	Canon EOS camera.	Indian sign language.	500 video samples.

Author/Year	Pros	Cons
Gurbuz et al. 2021	Contactless sensing, environment independent	The author cannot perform a comparative analysis with the existing method.
Hassan et al. 2019	Require less computation time.	Robustness needs to be improved.
Tripathi and Nandi 2015	New ISL continuous dataset presented.	Misclassification for a similar type of gesture.

3.5. Object Detection in SLR: The Role of YOLO

*3.5.1. **Introduction***

YOLO (You Only Look Once) is a state-of-the-art, real-time object detection algorithm based on deep neural networks (CNN), widely recognized for its remarkable speed, accuracy, and efficiency in detecting multiple objects within a single image or video frame. YOLO treats object detection as a regression problem, in which regression is used to directly generate candidate photos, their categories, and confidence indices. The non-maximum suppression technique and a threshold of the confidence rate are used to ultimately determine the detection outcome [35].

*3.5.2. **YOLO Architecture***

The YOLO (You Only Look Once) object detection system introduces a unified framework that reframes object detection as a single regression problem. Unlike traditional methods that treat detection as a multi-stage pipeline—such as R-CNN and Fast R-CNN [36] YOLO directly predicts bounding box coordinates and class probabilities from the full image in a single evaluation, achieving real-time speed with competitive accuracy [37].

I. Unified Detection Framework

YOLO divides the input image into an **S × S grid**, where each grid cell is responsible for detecting objects, whose centers fall within it [37]. For each grid cell, the model predicts:

1) **B bounding boxes**, each represented by: Center coordinates (x,y), Width and height (w,h), and a confidence score:

$$Confidence = \Pr(Object) \times IOU_{pred}^{truth} \quad (1)$$

- $\Pr(Object)$: The probability that an object exists in that grid cell.
- IOU_{pred}^{truth}: The intersection-over-union between the predicted bounding box and the ground truth box.

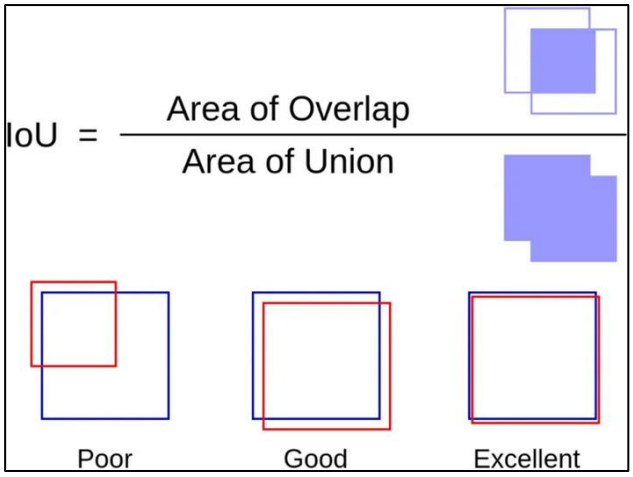


Figure 7: Visual representation of Intersection Over Union

This confidence score reflects both the probability that an object exists in the cell and the accuracy of the predicted bounding box. If no object exists in the cell, the confidence score is trained to be zero. Otherwise, it should equal the intersection over union (IOU) between the predicted box and the ground truth box [37].

2) **C class probabilities**, the probability that the object belongs to class i, given that an object exists in the grid cell, representing:

$$\Pr(Class_i \mid Object), \quad i = 1, 2, \ldots, C \tag{2}$$

- i: These are the indices for all possible object **classes**.
- In Pascal VOC: C = 20 classes (e.g., person, car, dog, etc.)
- In COCO: C = 80 classes
- In custom datasets, C can be any number depending on the object categories

These probabilities indicate the conditional class distribution, assuming an object is present in the cell [37]. All predictions from each cell are combined into a final output tensor of shape:

$$S \times S \times (5B + C) \tag{3}$$

- $S \times S$: grid size
- B: Bounding boxes with confidence scores
- C: Class probability

This compact tensor encodes all bounding boxes, confidence scores, and class distributions in a single forward pass, enabling YOLO to perform fast inference and global reasoning across the image [37].

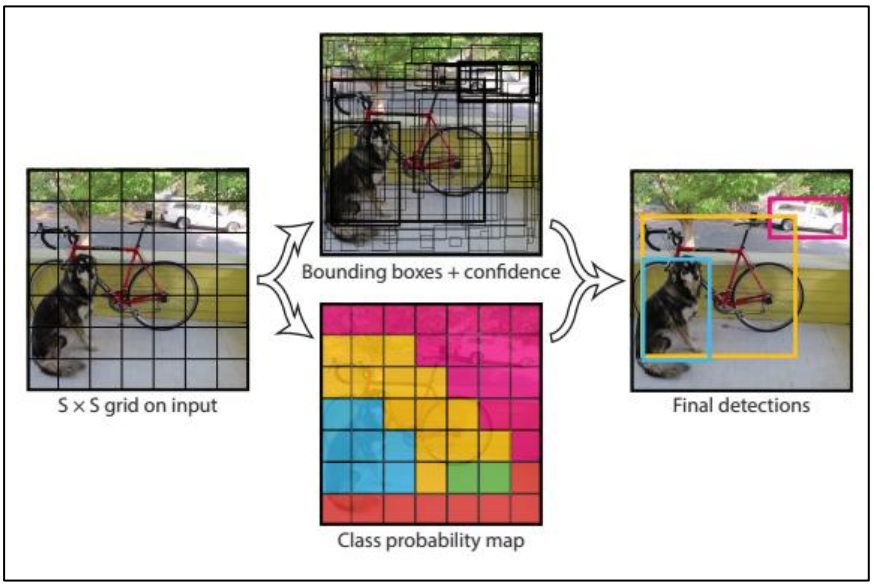


Figure 8: YOLO Object Detection Progress [31]

II. Architectural Components

YOLO's architecture consists of three main components: the **backbone**, **neck**, and **head**.

1) **Backbone:** The backbone is a convolutional neural network that extracts feature maps from the input image. The original YOLO model uses a custom CNN consisting of:

   - **24 convolutional layers**, inspired by GoogLeNet, using:
     - 1×11 \times 11×1 convolutions for dimensionality reduction,
     - 3×33 \times 33×3 convolutions for feature learning.
   - Followed by **2 fully connected layers**.

Unlike Fast R-CNN which uses pre-trained networks like VGG [36], YOLO trains this backbone from scratch, optimizing it specifically for detection performance.

2) **Neck**: Although the concept of a "neck" is not formally defined in YOLOv1, there exists a transition block—the final convolutional feature maps are flattened and passed through the fully connected layers. This enables the model to capture global spatial relationships and prepare the data for structured output.

3) **Head**: The detection **head** outputs predictions for each grid cell:

- Bounding box coordinates (x,y,w,h)
- A **confidence score**, defined as in equation (1)
- Conditional class properties defined as in equation (2)

At inference, the **final score** for each class per box is computed by:

$$\Pr(Object) \cdot IOU_{pred}^{truth} = \Pr(Class_i|Object) \cdot \Pr(Object) \cdot IOU_{pred}^{truth} \quad (4)$$

To eliminate duplicate predictions, **Non-Maximum Suppression (NMS)** is applied.

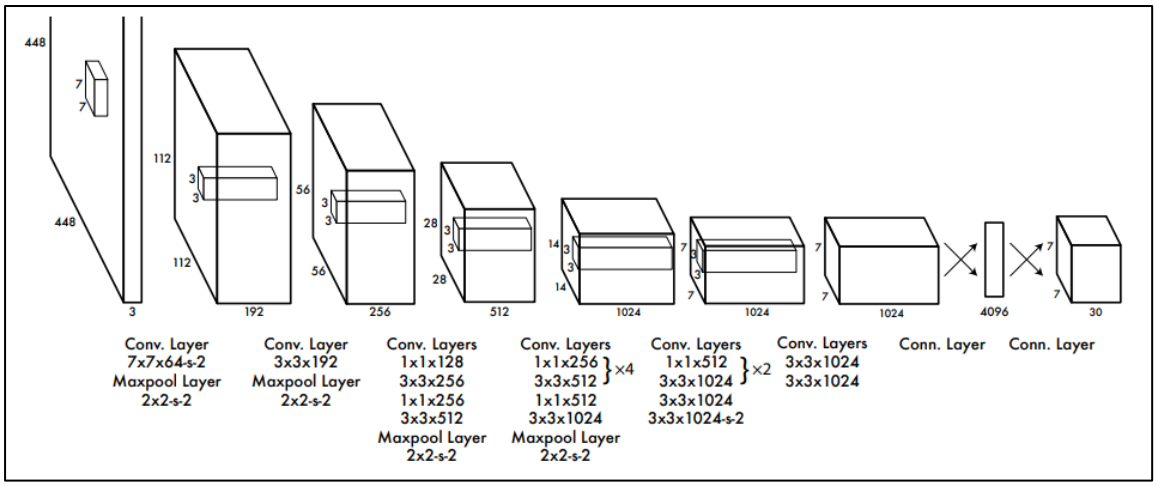


Figure 9: Original YOLO Architecture [38]

YOLO is trained end-to-end to perform object detection directly from images. The training process begins with pretraining the convolutional backbone on the ImageNet 1000-class dataset. Specifically, the first 20 convolutional layers (as shown in the original architecture diagram) are followed by an average pooling layer and a fully connected layer, achieving a top-5 accuracy of 88% on the ImageNet 2012 validation set—comparable to GoogLeNet [37], [39].

III. Model conversion for detection

After pretraining, the network is converted for object detection by:

- Adding four additional convolutional layers and two fully connected layers with randomly initialized weights [40].
- Increasing the input resolution from 224×224 to 448×448 to preserve fine-grained visual information necessary for detection.
- Modifying the final output layer to predict both class probabilities and normalized bounding box coordinates.

The bounding box coordinates (x,y,w,h) are normalized between 0 and 1:

- (x,y) are relative to the grid cell.
- (w,h) are normalized by the full image size.

IV. Activation and loss function

YOLO uses the leaky ReLU activation function in all layers except the last one. This function outputs the input directly if it's positive, and scales it by 0.1 if it's negative:

$$\phi(x) = \begin{cases} x & \text{if } x > 0 \\ 0.1x & \text{otherwise} \end{cases} \quad (5)$$

The final layer uses a linear activation to produce continuous outputs for bounding box coordinates and class probabilities. During training, YOLO minimizes a sum-squared error loss function that combines several parts:

$$L = \lambda_{coord} \cdot Localization\ Loss + Confidence\ Loss + Classification\ Loss \quad (6)$$

- **Localization loss:** Measures the error in predicted box coordinates (x, y, width, height).
- **Confidence loss**: Compares predicted object confidence with the actual presence of an object.
- **Classification loss:** Penalizes incorrect class predictions, only if an object is present.

The full loss function is:

$$
\begin{aligned}
&\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2\right] \\
&+\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i}\right)^2\right] \\
&+\sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i\right)^2 \\
&+\lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i\right)^2 \\
&+\sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{ classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}
\tag{7}
$$

Where:

- 1_{ij}^{obj} is 1 if the jth bounding box predictor in cell i is "responsible" for an object.
- 1_{ij}^{noobj} is 1 if no object is present in that box.
- (x_i, y_i, w_i, h_i) are the ground truth box coordinates.
- $\left(\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i\right)$ are the predicted box coordinates.
- C_i and $\hat{C}_i$ are the ground truth and predicted confidence scores.
- $p_i(c)$ and $\hat{p}_i(c)$ are the true and predicted class probabilities for class c.

Weighting coefficients:

- $\lambda_{\text{coord}} = 5$: Increases focus on bounding box coordinate accuracy.
- $\lambda_{\text{noob }j} = 0.5$: Reduces the influence of background grid cells that contain no objects.

This loss formulation ensures:

- Only the bounding box with the highest IOU is responsible for an object.
- The model specializes in box predictors and emphasizes localization over background confidence.
- Class prediction is only penalized in grid cells containing objects.

*3.5.3. **YOLO Versions and Sizes***

Initially introduced by Redmon et al. in 2015 [37], YOLO revolutionized object detection by adopting a single-stage detection architecture, distinguishing it from traditional two-stage detectors such as Faster R-CNN. Since its inception, the algorithm has undergone several enhancements, leading to the development of multiple versions including YOLOv2, YOLOv3, and YOLOv4 [41], until YOLOv11 nowadays, as well as lightweight adaptations such as YOLO-LITE, designed for constrained environments. This ongoing evolution demonstrates YOLO's adaptability to both research and industry needs, making it a foundational component in the field of modern computer vision [41] and well-suited for real-time Sign Language Recognition.

The differences between YOLO model sizes stem from variations in architectural depth, width, and computational complexity, which significantly impact accuracy, speed, and suitability for deployment. Larger models such as YOLOv5x or YOLOv8x consist of more layers and parameters, allowing them to capture complex patterns and perform better on intricate datasets, but at the cost of increased inference time and higher hardware demands, making them ideal for accuracy-critical applications [42]. Conversely, smaller models like YOLOv5n or YOLOv8n are optimized for speed and low-power usage, making them suitable for real-time or edge deployments, albeit with a modest trade-off in accuracy. These architectural differences are governed by various hyperparameters—settings such as the number of layers, filters per layer, kernel sizes, and learning rates—which are defined before training and significantly influence how the model learns. Larger models inherently involve more hyperparameters, which give them greater learning capacity but also make them more computationally intensive and sensitive to tuning. As a result, while large models offer enhanced performance, they require more careful configuration and resource investment to avoid issues like overfitting or underfitting [42].

The following chart shows a comparison between YOLO versions and sizes in terms of latency (speed of inference) and accuracy.

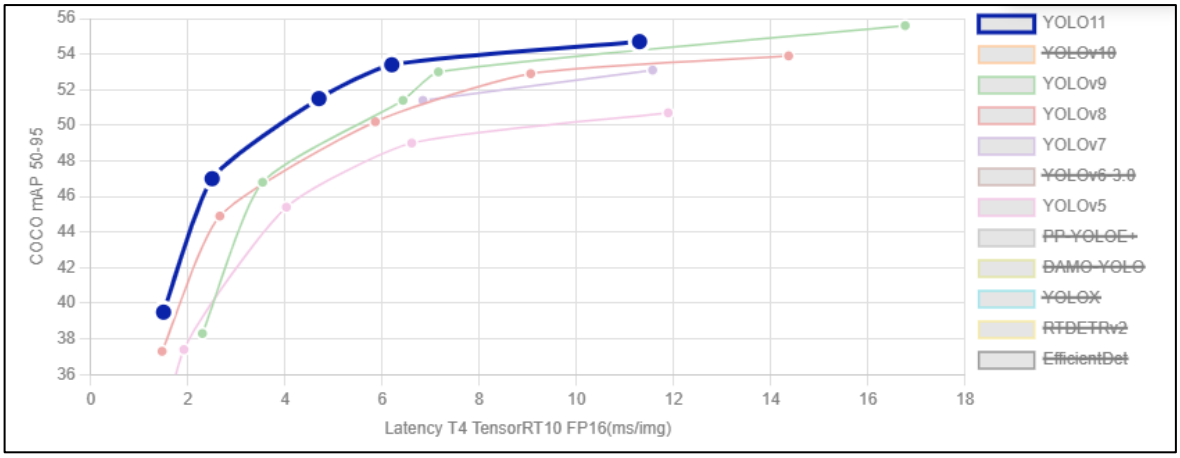


Figure 10: YOLO Versions and Sizes Chart [43]

*3.5.4. **Training YOLO for SLR***

For SLR applications, the architecture is typically modified to focus on hand and body part detection, with specialized attention to small gesture features.

Effective YOLO training for SLR requires [44]:

- Specialized datasets like WLASL (Word-Level American Sign Language)
- Data augmentation techniques (e.g., random crops, lighting variations)
- Transfer learning from pretrained weights
- Optimization of anchor boxes for hand shapes
- Custom loss functions balancing localization and classification accuracy

3.6. YOLO vs. other CNN based Detectors

Object detection has evolved significantly with the development of various deep learning-based approaches. Among the most influential methods are R-CNN, Faster R-CNN, and YOLO [36], [37]. These models represent different design philosophies, each with strengths and trade-offs in terms of accuracy, speed, and architecture complexity.

3.6.1. Architectural differences

- **R-CNN**: Extracts ~2,000 region proposals using Selective Search. Each region is independently passed through a CNN for classification and bounding box regression[36].
- **Faster R-CNN**: Integrates a Region Proposal Network (RPN) into the CNN backbone, greatly reducing inference time [36].
- **YOLO**: Processes the entire image in a single forward pass, dividing it into an S×S \times S×S grid, with each cell directly predicting bounding boxes and class probabilities.

Table 4: YOLO, R-CNN, and Faster R-CNN architectural differences

Component	R-CNN	Faster R-CNN	YOLO
Pipeline	Multi-Stage	Two-Stage	Single-Stage
Regional proposals	Selective Search (external)	Region Proposal Network (RPN, internal)	No proposals; direct grid-based prediction
Feature sharing	No	Yes (shared backbone and RPN)	Yes (single CNN for all predictions)
Post-preprocessing	SVM + NMS	Softmax + NMS	Confidence threshold + NMS

- R-CNN: Multi-Stage: region proposal → feature extraction → classification
- Faster R-CNN: Two-Stage: RPN → feature extraction + classification
- YOLO: Single-Stage: image → predictions

*3.6.2. **Speed, accuracy and performance***

One of YOLO's primary contributions is its **real-time performance**, it achieves significant speed gains by eliminating the need for region proposals and sharing computation across the entire image. This makes it suitable for applications like robotics, autonomous driving, and video surveillance [31].

Table 5: YOLO, R-CNN, and Faster R-CNN speed, accuracy, and performance comparison [31]

Model	Speed (FPS)	mAP on PASCAL VOC 2007	Notes	Best use case
R-CNN	~0.05 FPS	66.0%	Very slow due to external proposals and per-region processing	Academid research, where speed is not critical
Faster R-CNN	~7 FPS (GPU)	73.2%	Faster, but still not real-time	Application needing high accuracy
YOLO	45 FPS (YOLOv1)	63.4%	Real-time detection	Real time systems

*3.6.3. **Global vs local context***

- **YOLO** looks at the **entire image during training and test time**, enabling it to learn global context and reduce false positives on background objects.
- **R-CNN-based models** focus on **local regions**, which may result in more accurate localization but can miss contextual clues that differentiate similar-looking objects.

4. Projects Realistic Constraints

Setting realistic limits is essential in project management to prevent overwork, optimize resource utilization, and maintain both budget and schedule. They also support achievable goal setting and significantly increase the likelihood of successful project completion.

1. Performance

The system focuses on efficiently and accurately translating sign language into text in real time. Since it relies on recognizing hand gestures from continuous video, smooth capture, low error rates, and fast text output are essential. Given the limited processing power of Raspberry Pi 5, performance optimization is critical. Both the software and machine learning model are carefully tuned to balance speed and accuracy within these hardware constraints. The system should achieve the following parameters:

- Consistent detection accuracy of 70% or above.
- Processing latency of 1 second or less.
- Video data rate minimum at 2 – 3 frames per second.

2. Functionality

The device is designed to fulfill core functional requirements essential for real-time sign language translation using a Raspberry Pi. It captures hand gestures through a connected camera (autofocus Arducam camera), processes the input using a trained machine learning model, and displays the translated text on an OLED LCD screen. This involves continuous video frame acquisition, the application of preprocessing techniques, and accurate gesture classification into predefined text. Each component operates in sync to transform raw input into meaningful output, supporting the system's goal of enabling seamless and accessible communication.

3. Economics

The system is designed as a low-cost assistive device for the hearing-impaired community, focusing on affordability, essential functionality, and reliable performance within a limited budget. Cost Targets:

- Retail price: Must not exceed 200 JD, including all hardware components such as the Raspberry Pi 5, camera, LCD screen, power supply, and support electronics.
- Annual maintenance and software updates: Must remain under 3,000 JD to ensure long-term affordability and sustainability.

4. Energy

The system is designed to operate with low power consumption, aligning with the energy-efficient nature of the Raspberry Pi platform.

- Operation is maintained within the limits of a standard 5V power supply.
- Power supply delivered via a USB adapter with current draw kept around 3-5A to ensure stable performance.
- The system will have an average power consumption of 10 W.

5. Environmental

The device is engineered for low power consumption, reflecting the energy-efficient design of the Raspberry Pi platform. To support this, both hardware configurations and software operations are optimized to eliminate unnecessary energy usage while preserving full functionality. A minimum of 60% of the SignBridge Translator components— including the Raspberry Pi module, camera, and display—will be designed for easy disassembly to facilitate repair, enable reuse in similar devices, and help reduce electronic waste.

6. Sustainability

- The system is built with modular, energy-efficient components such as the Raspberry Pi and PiCamera, ensuring long-term usability and easy upgrades or replacements.
- It operates on low power and supports the use of rechargeable batteries, minimizing environmental impact and reducing electronic waste.
- Many of its electronic parts are widely available and can be reused or repurposed in other applications, supporting sustainability in a circular economy.
- The focus on durability, repairability, and open-source software ensures the system remains functional and sustainable over time.

7. Health and Safety

The product is developed in accordance with UL and CE safety standards to reduce risks such as electrical shock, overheating, and electromagnetic emissions. It is housed in a low-voltage, insulated enclosure designed to prevent contact with internal circuitry and to manage heat during extended use. The exterior is smooth and free of sharp edges, ensuring it is safe and accessible for users of all ages and physical sensitivities, including children and individuals with disabilities. The system also follows ANSI C95.1 guidelines for safe electromagnetic exposure and will be tested to confirm compliance before release.

8. Ethical

The product aligns with ethical standards by serving as an assistive tool that enhances communication for individuals with hearing and speech impairments, promoting inclusion and equal access. No conflicts with professional Codes of Ethics are anticipated, as the system is designed to benefit society without causing harm. It does not store or transmit personal data, which helps protect user privacy and avoids ethical concerns related to data misuse. Additionally, the use of open-source software and accessible hardware supports transparency, fairness, and responsible development practices.

9. Legal

The system is developed with legal and ethical compliance in mind, especially in view of potential distribution or commercial use. Efforts are made to avoid infringing on patents, copyrights, or trademarks related to hardware, software, and machine learning models. All pre-trained models, datasets, and open-source code are properly licensed and attributed. If any user data is involved, the system adheres to relevant data protection laws in the target region to ensure privacy and legal compliance.

10. Maintainability

The system is built with maintainability in focus, ensuring long-term operational reliability and ease of updates throughout its service life. Its hardware is modular, allowing the components—such as the camera, OLED LCD screen, or Raspberry Pi board—to be easily replaced or upgraded as needed. On the software side, a structured and well-documented codebase facilitates efficient troubleshooting, smooth implementation of updates, and the integration of enhancements, including the retraining of the model with new gesture data. This approach supports ongoing functionality and adaptability over time.

11. Manufacturability

Components such as the Raspberry Pi, 16MP camera module, 128x64 OLED LCD, and supporting electronics are selected for low cost, global availability, and compatibility with open-source platforms. The layout is compact and modular, enabling easy assembly with basic tools. Standard connectors like GPIO headers and ribbon cables simplify integration and reduce errors. This design supports scalable production and consistent quality through straightforward testing and clear documentation.

12. Operational

The device is designed to maintain reliable performance across a variety of real-world environments, including homes, classrooms, and public spaces. It operates effectively within standard indoor temperature ranges (0C to 50C) without experiencing performance degradation and is resilient to moderate humidity levels, ensuring stability in slightly moist conditions. The physical build is engineered to withstand common disturbances such as minor shocks or vibrations from handling and transport.

13. Political

The product will comply with international regulations for safety, electromagnetic compatibility, and environmental standards, with certifications such as CE and FCC obtained as needed. It will follow applicable data protection laws, including the GDPR, to ensure user privacy. All software, models, and datasets will be properly licensed in accordance with intellectual property laws such as the DMCA. The system has no affiliation with political, governmental, or union organizations unless required for certification.

14. Reliability

Reliability refers to the system's ability to perform its intended functions accurately and consistently over time without unexpected failures or significant performance degradation. For this sign language translation device, reliability means it can continuously process video input, recognize gestures correctly, and display accurate text output without interruptions or errors during prolonged use. The system is designed to achieve a reliability of 95% over four years, ensuring dependable performance that users can trust for everyday communication.

15. Availability

The system is designed to operate consistently over extended periods with minimal interruptions, achieving a projected operational readiness of 97% during regular use. This is supported by reliable components such as the Raspberry Pi, camera module, and OLED LCD screen, along with stable computer vision algorithms that maintain accurate sign language translation. Extensive testing - including failure rate analysis and Mean Time Between Failures (MTBF) evaluation - will be performed, with the MTBF expected to reach 1,000 hours under continuous operation. This ensures the system remains functional and responsive with minimal downtime throughout long-term use.

16. Social and Cultural

Social and cultural constraints involve the device's benefits, risks, and acceptance. It promotes inclusivity by enabling communication between users and non-users of sign language. However, there are risks related to its accuracy. Misinterpretations could lead to frustration and communication barriers. The device must also be easy to use and reliable, ensuring that users feel confident in its ability to accurately translate sign language into spoken language.

17. National and Global

- The sign language translation system is developed in accordance with national standards for electronic safety and assistive technologies, supporting safe and reliable use in local contexts.
- The design follows safe voltage levels and uses enclosed components to prevent electrical hazards, meeting essential safety codes.
- Where possible, the project aims to align with international standards such as CE.
- The use of standard, widely accepted components helps ensure the system can be adapted to meet both national and international regulations.

5. Alternative Designs

5.1. SignForDeaf

SignforDeaf has developed tools to address communication challenges faced by hearing-impaired and deaf individuals in accessing information and services [45]. It has various plugins and systems in place to help accomplish that. The Web Sign Language Plugin lets website text be clicked for instant sign language translation. The Video Sign Language Plugin uses artificial intelligence to translate video subtitles into sign language in real time. The Double-Sided Communication System helps communication between hearing-impaired individuals and non-signers by converting text/voice into sign language and vice versa.

SignforDeaf's major disadvantage is that it requires a personal computer or laptop to download and run it, which makes it impractical for on-the-go tasks or short interactions. Additionally, it only supports broad applications, such as web and video translation, and lacks real-time face-to-face communication.

5.2. Gesture-to-Text Mobile Applications

Mobile apps that convert gestures into text help deaf individuals communicate by turning sign language into readable text in real time [46]. These apps use AI to recognize hand gestures and translate them into text. The apps process sign language with the smartphone camera and show the text on the screen, making it easier for signers and non-signers to interact. An example of these apps is HandTalk. This app is handy in various settings, providing a portable and efficient way to bridge communication gaps.

Sign language translation apps face similar limitations as SignforDeaf. While most people today own smartphones, the requirement to download the app and take out the device for every social interaction—regardless of how brief—can be impractical. However, this technology offers greater flexibility than SignforDeaf, as it only requires a smartphone, making it more accessible for users in various settings.

5.3. Intelligent Gloves (IG)

Intelligent gloves are designed to recognize hand gestures corresponding to words and phrases in Sign Language. These gloves use sensors to record hand positions and movements and transmit the data to a central computer [47]. The computer processes the gesture data using an algorithm. When a match is identified, the associated word or phrase is spoken through a speaker. For example, **SignAloud** gloves are a specific implementation of this technology. Figure 5 shows one version of these gloves.



Figure 11: Intelligent Gloves for The Deaf

Some of IG's shortcomings are cost, inconvenience, and cultural factors. Since every deaf individual is required to purchase a pair of these gloves, the financial burden can be significant, especially considering the ongoing need for replacement or maintenance. Additionally, having to wear these gloves all the time to communicate can be inconvenient and may attract unwanted attention, which can be uncomfortable and undesirable for many users.

6. The Design Solution

To achieve our goal of bridging the gap in communication between hearing-impaired and spoken people, we implement the design of The Real-Time Sign Language Translation Unit. The system leverages YOLOv8-based hand detection and a Raspberry Pi 5 processing unit to translate sign language gestures into text in real time.

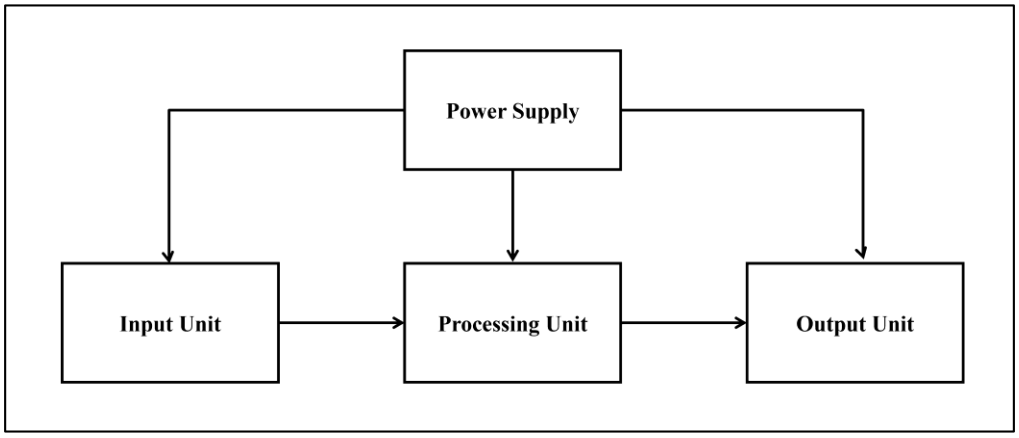


Figure 12: System Block Diagram

The system consists of three main parts, as shown in figure 6: the input device, a camera that captures live video of hand gestures; the processing unit, a Raspberry Pi 5 that processes acquired data; and the output device, an OLED LCD that displays output text. Annotated frames can also be viewed via OpenCV.

6.1. Hardware Components

6.1.1. ***Processing Unit (Raspberry Pi 5)***

The Raspberry Pi 5 is a compact computing platform designed to support a wide range of development, automation, and educational applications. It offers enhanced performance and expanded capabilities, making it well-suited for both prototyping and deployment in embedded systems. The Raspberry Pi 5 has the following specifications [48]:

- Broadcom BCM2712 2.4GHz quad-core 64-bit Arm Cortex-A76 CPU, with cryptography extensions, 512KB per-core L2 caches and a 2MB shared L3 cache
- VideoCore VII GPU, supporting OpenGL ES 3.1, Vulkan 1.2

- LPDDR4X-4267 SDRAM (2GB, 4GB, 8GB, and 16GB)
- 2 × 4-lane MIPI camera/display transceivers
- 5V/5A DC power via USB-C, with Power Delivery support
- Raspberry Pi standard 40-pin header

Though the Raspberry Pi 4 is a good option, Raspberry Pi 5 was chosen because it delivers a significant boost in both CPU and GPU performance. It offers a 2–3× increase in processing power compared to the Raspberry Pi 4 [48].

The Pi 5 offers LPDDR4X-4267 SDRAM in configurations ranging from 2GB to 16GB, with the 4GB model selected for this project as it balances cost and performance for real-time video processing and translation.

In this project, Raspberry Pi 5 will be the core computational unit for the Sign-to-text Translation System. It will process live video captured by the connected camera module to identify gestures, run computer vision algorithms and deep learning models for gesture recognition, and manage the output to convert recognized gestures into text. Figure 8 shows the Raspberry Pi and its ports.

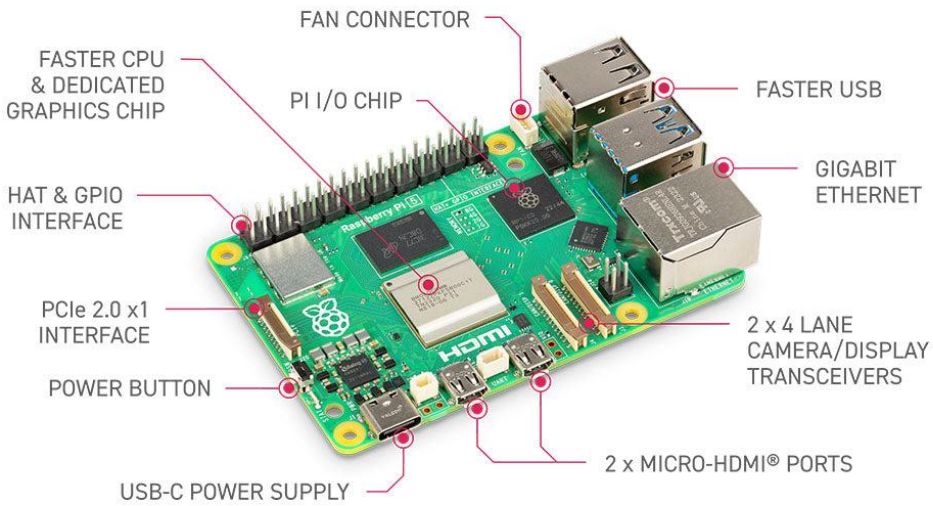


Figure 13: Raspberry Pi 5 [49]

6.1.2. ***Input Device (Arducam Camera Module 3)***

The Arducam Camera Module V3, based on the 16MP Sony IMX519 sensor - which is shown in figure 6, was selected for this project due to its motorized autofocus feature, which ensures clear image capture regardless of the user's distance from the camera. This capability is particularly desirable for real-time sign language recognition, where varying distances can affect the accuracy of gesture detection. As shown in Table 1, the Arducam Camera Module V3 offers improved specifications compared to the Raspberry Camera Modules v2 and has similar specifications to Raspberry Camera Module 3, including more versatile capture rates and resolutions, making it well suited for this application. Among the available options, 30 FPS was found to be a good capture rate, offering a balance between smooth motion capture and processing efficiency [50], [51].

ArduCam Camera Module v3 is fully compatible with the Raspberry Pi 5, connecting via the 4-lane MIPI camera/display transceiver. This built-in integration allows for efficient video capture and processing, leveraging Raspberry Pi 5's computational capabilities to handle the demands of real-time image recognition and translation tasks.



Figure 14: ArduCam Camera Module v3

Table 6: Camera model specifications [52], [53]

Specifications/Model	Raspberry Camera Module v2	Raspberry Camera Module v3	ArduCam Camera Module v3
Net price	30 JD	30 JD	39 JD
Still resolution	8 Mp	11.9 Mp	16 Mp
Video modes	1080p47, 1640 × 1232p41 and 640 × 480p206	2304 × 1296p56, 2304 × 1296p30 HDR, 1536 × 864p120	1080p 30 FPS, 720p 60FPS
Sensor resolution	3280 × 2464 pixels	4608 × 2592 pixels	4656×3496 pixels
Focus	Adjustable	Motorized	Autofocus
Depth of field	Approx 10 c m to ∞	Approx 10 cm to ∞	10cm to ∞
Focal length	3.04 mm	4.74 mm	4.28 mm

*6.1.3. **Output Device (128x64 OLED LCD)***

Processed outputs are displayed on a 128x64 OLED LCD module connected to the Raspberry Pi 5 via the I²C communication protocol. The module comes with four pins by default—VCC (5V), GND, SDA (data), and SCL (clock)—making it easy to integrate while keeping wiring minimal. This setup reduces the number of GPIO pins required and maintains reliable text rendering. OLED's higher resolution supports clearer and more flexible output, allowing for more detailed and readable real-time translations.

Standard character LCDs, such as 16x2 or 20x4 modules, are limited to built-in ASCII character sets and do not natively support Arabic script, which requires right-to-left text rendering and contextual character shaping. These limitations make them unsuitable for displaying accurate and legible Arabic output. In contrast, 128x64 OLED LCD offers full graphical rendering capabilities, allowing for custom font integration and proper handling of Arabic text.



Figure 15: 128x64 OLED LCD

6.1.4. Additional components

I. 30W Type-C Charger:

The system uses a 30-watt Type-C charger as the primary power supply for the Raspberry Pi. This charger delivers a stable and sufficient current to support the continuous operation of the Raspberry Pi, which in turn powers all other connected components, including the camera and OLED LCD screen.

II. Camera Stand

A stable camera stand is employed to securely mount the Arducam camera. This component ensures the camera remains in a fixed, upright position to maintain consistent image alignment for accurate data capture and processing.

III. Raspberry Pi Official Case with Fan

The Raspberry Pi is enclosed in a case provided by Raspberry Pi, designed specifically for thermal regulation and board protection. The case includes a dedicated fan, with a maximum fan speed of 8000 RPM +/- 15% and maximum airflow of 2.79 CFM [54], that connects directly to the 5V fan header on the Raspberry Pi board, allowing for active cooling and maintaining safe operating temperatures. Heat sinks are attached to further help with heat management.



Figure 16: RaspberryPi 5 case with integrated fan

6.2. Software Components

This section outlines the software components used to run the detection pipeline. Each software component plays a specific role in handling image capture, object detection, display, and communication.

*6.2.1. **Operating system***

The system runs on Raspberry Pi OS (Bookworm or Bullseye) which has the following advantages:

- **Optimization and Compatibility**: One of the biggest advantages of using Raspberry Pi OS is that it's developed specifically by the Raspberry Pi Foundation to run on Raspberry Pi hardware. This means it offers out-of-the-box support for all the board's features, including GPIO pins, camera modules, video acceleration, and various add-ons. It's optimized for the Pi's specific architecture, so it tends to run more efficiently and reliably than general-purpose ARM distributions like Ubuntu or Arch Linux ARM, which often require manual tweaks or extra drivers to access full hardware functionality.

- **Community and Documentation**: Another strong reason to choose Raspberry Pi OS is the breadth of its community and the wealth of available documentation. Because it's the official and most widely used operating system for Raspberry Pi, most tutorials, troubleshooting guides, and project instructions found online are written with it in mind. The chances are high that to find step-by-step resources tailored to Raspberry Pi OS.

*6.2.2. **Python***

Python is a well-known high-level programming language, commonly utilized in areas such as automation, data science, AI, and web development [55], [56]. Its numerous powerful libraries have made it a leading programming language choice for artificial intelligence (AI) and machine learning.

Advantages of using Python in AI and machine learning:

- **Ease of** Use: Python's simple syntax and readability make it easy for beginners to learn. It includes powerful data mining tools and essential libraries like Pandas, NumPy, and TensorFlow, making it a go-to language for data science [55].

- Flexibility: Python is versatile, enabling software development, data analysis, and web application creation. Frameworks like Django and TurboGears enhance its usability for various tasks.

- Deep Learning: Python supports deep learning with libraries like TensorFlow and Keras, helping developers create neural networks [56].

- Community Support: Python's vast developer community actively contributes to its growth. Platforms like Python.org and Python Package Index provide extremely helpful resources.

6.2.3. Used Libraries

The system utilizes several key software libraries for its operation:

- **Picamera2:** This library interfaces with the Arducam camera module using the libcamera backend, providing camera control features including autofocus and real-time video frame capture. Captured frames are passed directly to the processing pipeline.

- **Ultralytics (YOLOv8)**: The core detection engine uses a custom-trained YOLOv8 model (.pt file) through the Ultralytics library. This provides real-time object detection capabilities, processing each frame from the camera to identify and classify hand gestures.

- **OpenCV (opencv-python)**: Computer Vision (CV) is an open-source Library written in C/C++ in Python, including numerous API used in applications of artificial intelligence, recognition systems, and computer vision [37]. OpenCV Handles multiple critical functions:

  - Image format conversion (RGB to BGR and vice versa)
  - Frame annotation (drawing bounding boxes and labels)
  - Real-time video display
  - User input handling for system control

- **Luma OLED:** The luma.oled library is used to interface with OLED displays over the I2C communication protocols. In this application, it facilitates communication between the Raspberry Pi and an SH1106-based OLED screen.

- **Arabic Text Rendering (PIL.ImageFont, arabic_reshaper, and bidi.algorithm):** A combination of libraries is used to properly render Arabic text on the OLED. ImageFont loads an Arabic-compatible font, arabic_reshaper adjusts characters into their correct contextual forms, and bidi ensures the text is displayed in right-to-left order.

6.2.4. ***Datasets***

In this work, two datasets were utilized: the **KArSL** dataset, which served experimentation and model comparison, and the SignBridge dataset, which was used to train the final model.

I. KArSL dataset

KArSL dataset, which is a specialized resource for Kuwaiti Arabic Sign Language developed by Hamzah Luqman, to train a robust sign language recognition model. The dataset comprises 75,000 RGB video samples captured at 256x256 resolution [57], enabling clear visualization of hand movements, finger positions, and motion transitions under varying lighting conditions. It features signs performed by three singers of different age groups and signing styles, contributing to its diversity and real-world applicability. The dataset includes both isolated signs and continuous signing sequences. KArSL has eleven chapters of the ArSL dictionary, such as Letters, Numbers, Health, Common Verbs, Family, Characteristics, Directions and Places, Social Relationships, In the House, Religion, and Jobs and Professions. The database was split into 2 datasets by the author: Training dataset (~80%) and Validation dataset (~20%) [57]. A subset of 3,000 frames was used from this dataset, representing 10 sign words corresponding to the digits 0 through 9.



Figure 17: KArSL dataset samples of the three signers for number (2) gesture.

II. SignBridge dataset

The final YOLO gesture detection model was trained using SignBridge database, which consists of 9 gestures of Arabic letters and words with a total of 838 samples. This database was created due to low detection accuracy when using the model trained on the KArSL database.

Dataset Creation and Capture Setup:

- **Hardware**: The system employs a Samsung Galaxy A33 smartphone, equipped with a 48 MP rear camera, for image acquisition.
- **Capture Method**: A custom Python script is used to capture images at 1.5-second intervals. Each image is stored in JPEG format with a resolution of 480×640 pixels.
- **Subjects**: The dataset features two different signers, each performing nine distinct hand signs.
- **Environments**: Data collection was conducted in two settings: an indoor environment with artificial lighting and an outdoor environment with natural daylight.

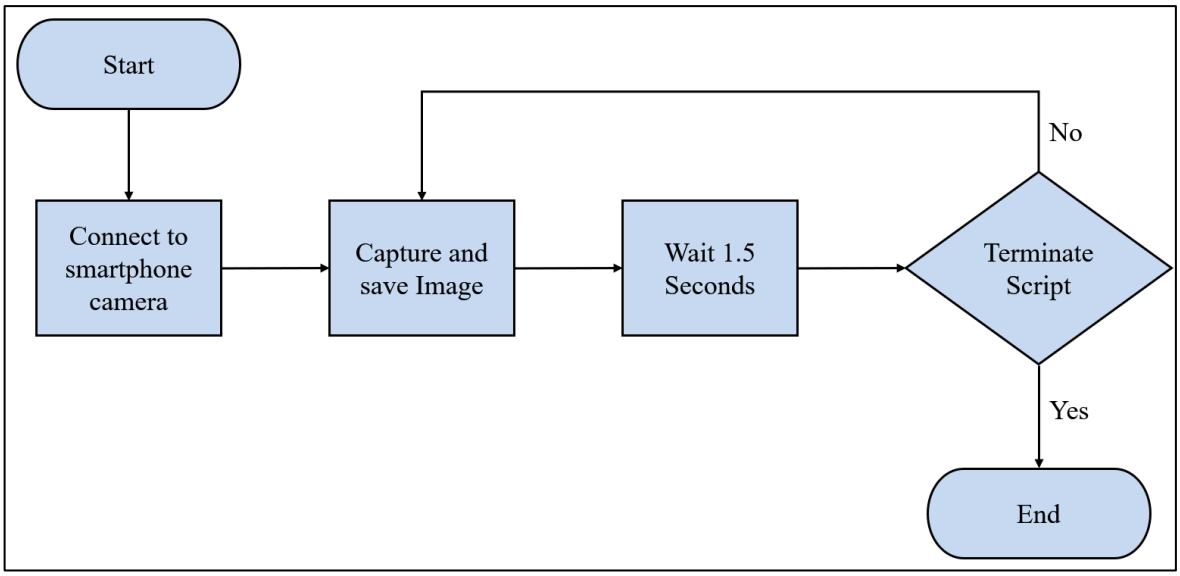


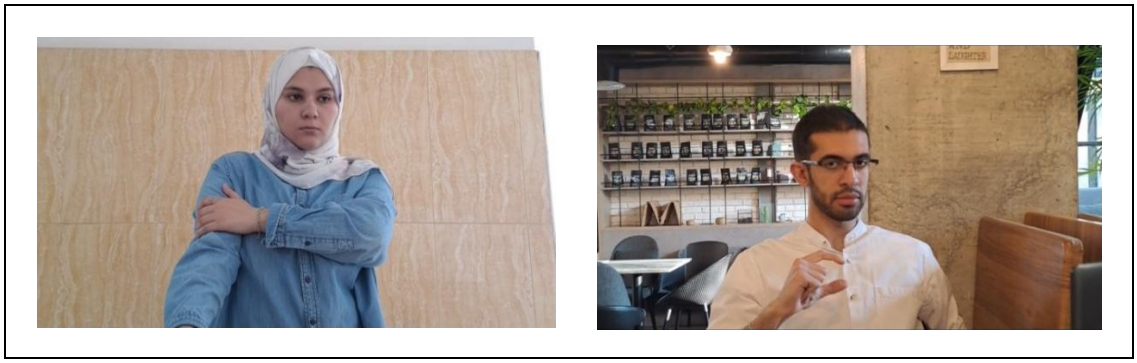
Figure 18: SignBridge dataset creation flowchart.

The database was divided into two datasets:

- The training dataset (85%): 643 images, 110 background (Unlabeled).
- The validation dataset (15%): 79 images, 6 background (Unlabeled).



(a) Indoor environment



(b) Outdoor environment

Figure 19: (a) and (b) SignBridge Dataset samples of authors for different gestures.

*6.2.5. **CVAT***

A labeled dataset is required to train the detection model, which is based on the YOLO architecture and relies on bounding box annotations. To meet this requirement, the Computer Vision Annotation Tool (CVAT) is employed. As an open-source and cost-effective platform, CVAT supports the creation of both bounding box and key point annotations. Bounding boxes are manually drawn and refined with high precision, allowing for accurate localization of hands and fingers—an essential factor in optimizing model performance. Furthermore, a customizable workflow is provided, and annotations are converted into the appropriate text format for model training and validation.

- **Labeling Procedure:**

1. Upload the compressed dataset file to CVAT.
2. Use bounding box tool to annotate each gesture with the right label.
3. Export annotations in Ultralytics YOLO detection format.

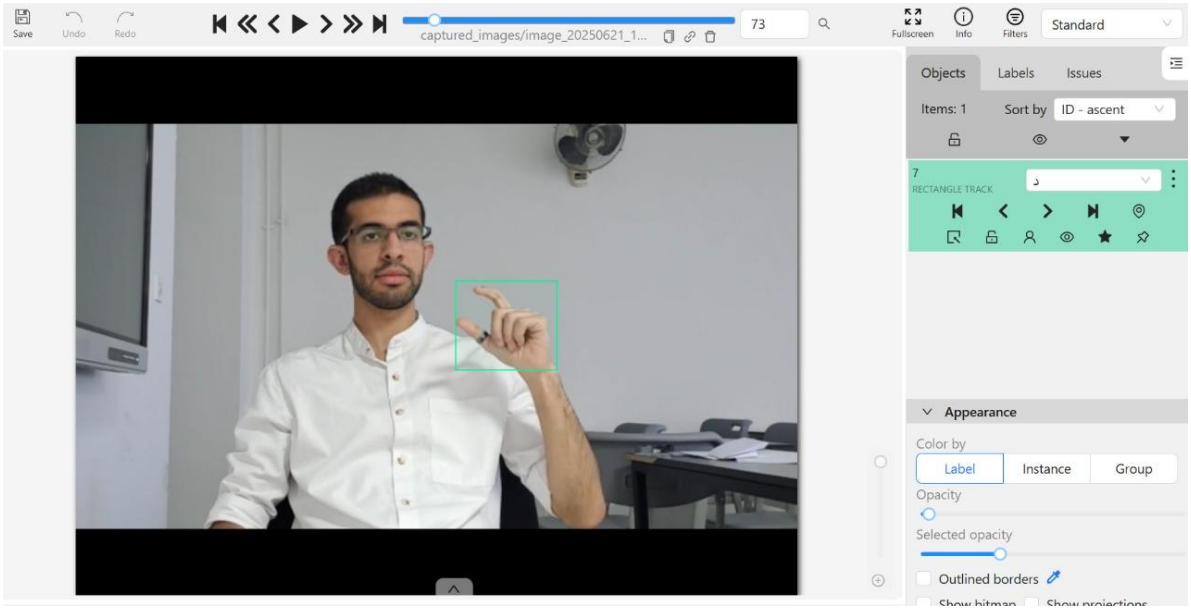


Figure 20: Dataset labelling using CVAT

6.2.6. *YOLOv8 detection model*

YOLOv8m is part of the Ultralytics YOLOv8 family, designed for fast and accurate object detection [58]. The architecture is modular and consists of three main components: the **backbone**, **neck**, and **head**.

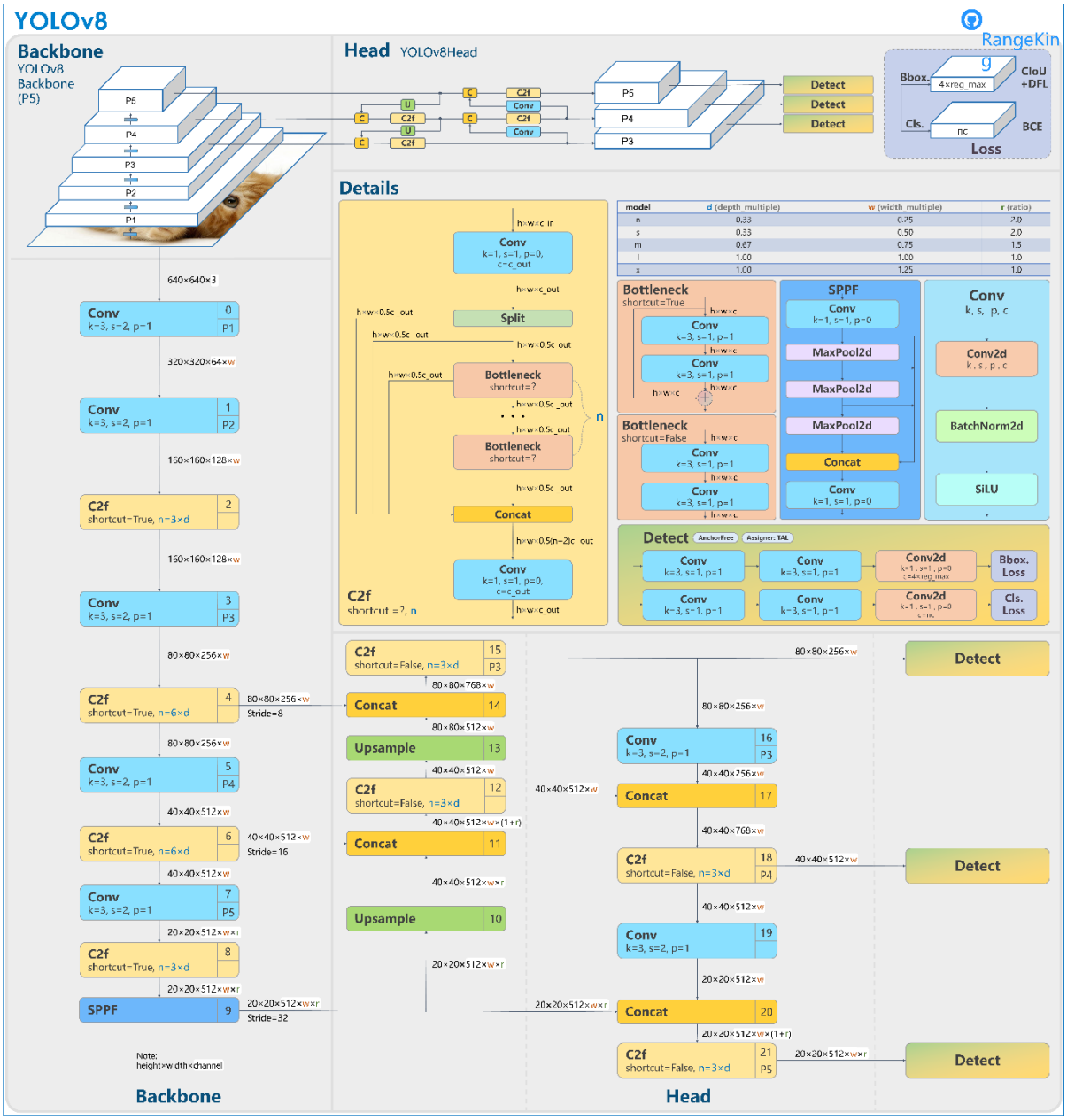


Figure 21: YOLOv8 Network Architecture [62]

I. Backbone

The backbone is responsible for extracting rich features from input images. YOLOv8m uses a modified version of CSPDarknet, which includes [59]:

- **Focus Layer**: Replaces initial convolution by slicing and concatenating, reducing computational cost.
- **C2f Blocks**: Efficient residual-like modules that reduce parameter usage while preserving representational power.
- **Convolutional Layers**: Each convolution is followed by BatchNorm and LeakyReLU.

Backbone Layer Composition shown in table 7:

Table 7: YOLOv8 backbone layers

Stage	Layers	Description
Stage 1	1 Convolutional	Focus & initial features
Stage 2	3 C2f blocks	Low-level features
Stage 3	6 C2f blocks	Mid-level features
Stage 4	6 C2f blocks	High-level features
Stage 5	3 C2f blocks	Semantic abstraction

II. Neck

The neck aggregates feature from different backbone levels to strengthen multi-scale detection. YOLOv8m uses a modified PAN (Path Aggregation Network), which includes [59]:

- **Top-down and bottom-up fusion** using up sampling and concatenation.
- **Feature Pyramid Fusion** that strengthens object detection for various object sizes.

Neck Layer Composition shown in table 8:

Table 8: YOLOv8 neck layers

Path	Operations
Top-down	Upsample → Concat → Conv (C2f)
Bottom-up	Conv → Concat → Conv (C2f)

III. Head

The detection head predicts bounding boxes, objectness scores, and class probabilities at different scales. It uses convolutional layers to [59]:

- Generate **bounding boxes**: center (x, y), width, and height.
- Predict **objectness score**: probability that an object exists.
- Predict **class scores** for each category.

Final output shape: (batch_size, num_predictions, (x, y, w, h, obj_conf, num_classes))

IV. Feature Extraction

Feature extraction relies heavily on convolutional operations across the backbone and neck. During training:

- **Low-level features** capture texture and color.
- **Mid- and high-level features** identify patterns, shapes, and object boundaries.

Software components used as shown in table 9:

Table 9: YOLOv8 software components

Component	Library	Purpose
Ultralytics YOLO	ultralytics	Model definition, training, and export
OpenCV	cv2	Image transformations and augmentations
Albumentations	albumentations	Data augmentation
PyTorch	torch	Training loop, optimization
NumPy	numpy	Matrix and tensor operations

V. Data Augmentation and Preprocessing

YOLOv8m applies extensive preprocessing and data augmentation techniques to improve generalization [60]:

- **Random Resizing and Cropping**: Changes the scale and composition of images.
- **HSV Color Shift**: Modifies hue, saturation, and brightness.
- **Horizontal Flip**: Simulates mirrored perspectives.
- **Normalization**: Pixel values are scaled between 0 and 1.

These transformations are automated by the Ultralytics train.py script using internally managed pipelines.

VI. Model sizes

To guide the selection of an appropriate YOLO model for real-time sign language recognition on embedded hardware, it is essential to consider the trade-offs between model accuracy, computational complexity, and inference speed. The following table 10 presents key performance metrics for different YOLOv8 model variants, including mean Average Precision (mAP), inference time on both CPU and GPU platforms, number of parameters, and floating-point operations per second (FLOPs). These figures provide a comparative overview to help balance model performance with hardware limitations, particularly when deploying on resource-constrained devices such as the Raspberry Pi 5.

Table 10: YOLOv8 detection model sizes performance [61]

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

6.3. YOLO Detection Model Versions Comparison

In our work, we evaluated the performance of YOLOv5m, YOLOv8m, and YOLOv9m on the Raspberry Pi 5 using the KarSL dataset and under identical experimental conditions to determine the most effective model version for sign language recognition. The training was done with the default YOLO configurations except for Image size = 256 and epoch = 25. The signer was positioned one meter away from the camera, under good lighting conditions, with minimal background disturbances. This setup was maintained throughout all experiments to provide clear visual input and reduce variability in model performance due to environmental factors.

The numbers (0 – 9) in the sign language were used for the evaluation between YOLO versions due to their simplicity and ease of testing. Figure 22 shows the environment the test was conducted in, and the tables display the experimentation results, with Table 11 showing the average confidence level for each number and Table 12 showing the processing time and latency on each version.

After analyzing the results, YOLOv8m outperformed the other versions in terms of detection accuracy generalization capability, making it the best choice for recognizing sign language gestures.

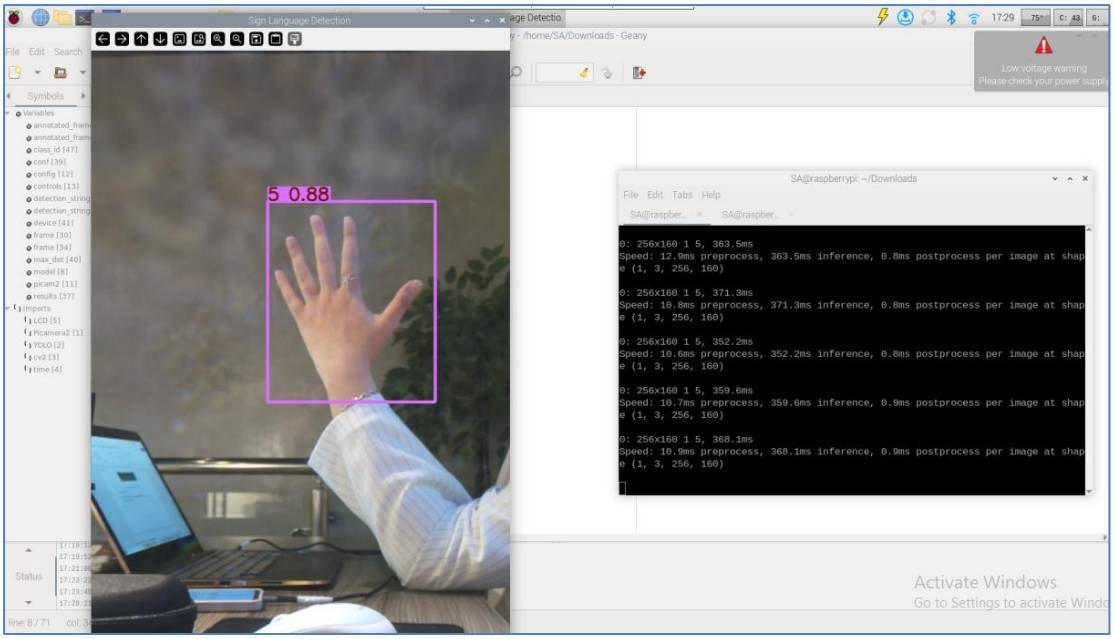


Figure 22: YOLO versions testing environment

Table 11: YOLO detection model versions accuracies

Version/Number	0	1	2	3	4	5	6	7	8	9
YOLOv-5M	-	-	-	-	0.3	0.42	-	0.81	0.32	0.64
YOLOv-8M	0.6	0.85	0.85	0.84	0.85	0.89	0.6	0.75	0.88	0.9
YOLOv-9M	0.8	0.88	0.8	0.81	0.72	0.95	0.8	0.7	0.9	0.82

Table 12: YOLO detection model versions performance

Version/Data	Processing Time	Latency
YOLOv-5M	310 – 370 ms	1s
YOLOv-8M	350 – 400 ms	1s
YOLOv-9M	380 – 430 ms	1s

6.4. YOLOv8-M Detection Model Training

This section outlines the complete training process of the YOLOv8m model tailored for Arabic sign language recognition. It covers the training environment setup, selected hyperparameters, and model configuration, followed by a detailed explanation of the training workflow. The training process leverages a custom dataset developed to improve detection accuracy for hand gestures and is optimized for deployment on resource-constrained hardware. Results from the training, including performance metrics and convergence behavior, are also presented to validate the effectiveness of the trained model.

6.4.1. ***Training Environment and Hardware Specifications***

The YOLO models were trained on a personal computer, as the Raspberry Pi lacks the computational resources required for training deep learning models. The Raspberry Pi is primarily used for running pre-trained models in real time, while all model development and training were conducted externally to accommodate the processing demands. The personal computer had the following specifications:

- Processor: Intel® Core™ i5-12400F (6 cores, 12 threads, up to 4.4 GHz)
- Graphics: NVIDIA® GeForce RTX™ 4060 with dedicated 8GB GDDR6 memory
- Memory: 16GB DDR4 RAM
- Storage: 1TB Kingston NV2 M.2 NVMe SSD (Model: SNV2S/1000G)

In order to accelerate the training process and handle the computational demands of deep learning more efficiently, the training was performed on the RTX 4060 GPU rather than relying on the CPU. Utilizing the GPU significantly reduced training time by enabling parallel processing of large batches.

*6.4.2. **Training Configuration and Hyperparameters***

The training configuration for the YOLO model was defined prior to the start of the learning process, specifying the conditions under which the model would be exposed to the dataset. It outlined key aspects of the training procedure, including how the data would be handled, how the model would update during training, and how long the process would run. During development, different parameters were tested to evaluate their effect on training behavior and consistency.

After experimentation, the best parameters for training using the SignBridge database were the following:

- Image size: 480
- Epochs: 30
- Batch size: 8
- Image size = 480
- Optimizer: Adam
- Initial learning rate (lr0): 0.001
- Final learning rate factor (lrf): 0.01
- Weight decay: 0.0005
- IoU threshold: 0.7
- Augmentation: Enabled
- Auto augmentation strategy: RandAugment

*6.4.3. **Training Flow work***

The training workflow in for the YOLOv8m gesture detection model, as illustrated in the flowchart in figure 23, begins by loading a custom dataset configuration file (Modified.yaml), which defines the classes and data paths for Arabic sign language gestures. The model is initialized using pretrained YOLOv8m weights (yolov8m.pt), leveraging transfer learning to enhance convergence and performance on the relatively small SignBridge dataset. Key hyperparameters such as learning rate, batch size, and image size are configured before entering the training loop.

Training proceeds over a maximum of 30 epochs, during which both training and validation phases are executed iteratively. After each epoch, model performance metrics (e.g., loss, mAP, precision) are logged, and early stopping criteria are checked to determine if training should terminate prematurely due to performance plateauing. If not halted early,

training continues until the final epoch is reached. Upon completion, the model's final weights are saved for deployment. This pipeline ensures efficient model development while monitoring for overfitting or diminishing returns during training.

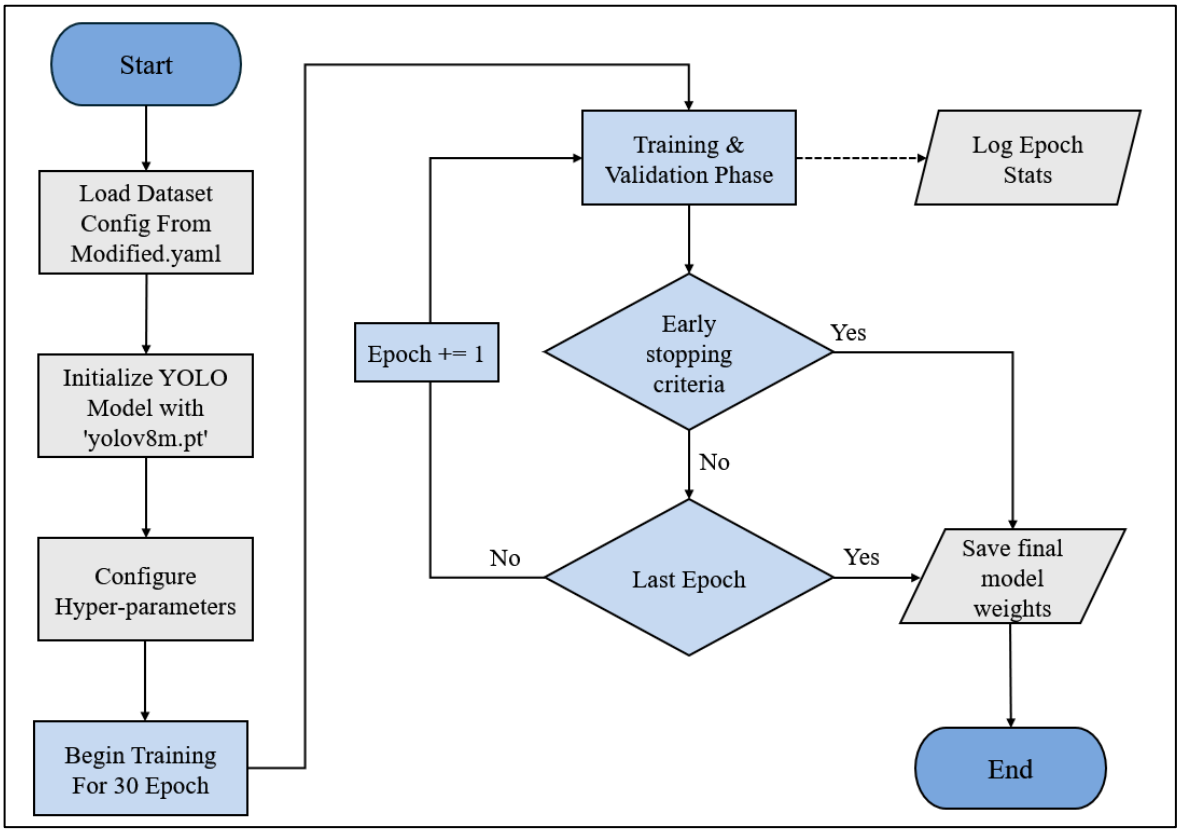


Figure 23: YOLOv8m detection model training flowchart

*6.4.4. **Training Results***

To determine the optimal training duration, the YOLOv8m model was trained across three different epoch configurations—20, 30, and 40 epochs. Among these, the 30-epoch training cycle was selected as it provided the best balance between convergence, validation performance, and training efficiency without signs of overfitting.

The training and validation curves shown in Figure 24 reflect the performance progression of the YOLOv8m model trained on the custom Arabic sign language dataset. Over 30 epochs, there is a consistent downward trend in training losses—specifically box loss, classification loss, and distribution focal loss (DFL)—indicating improved bounding box localization and classification accuracy.

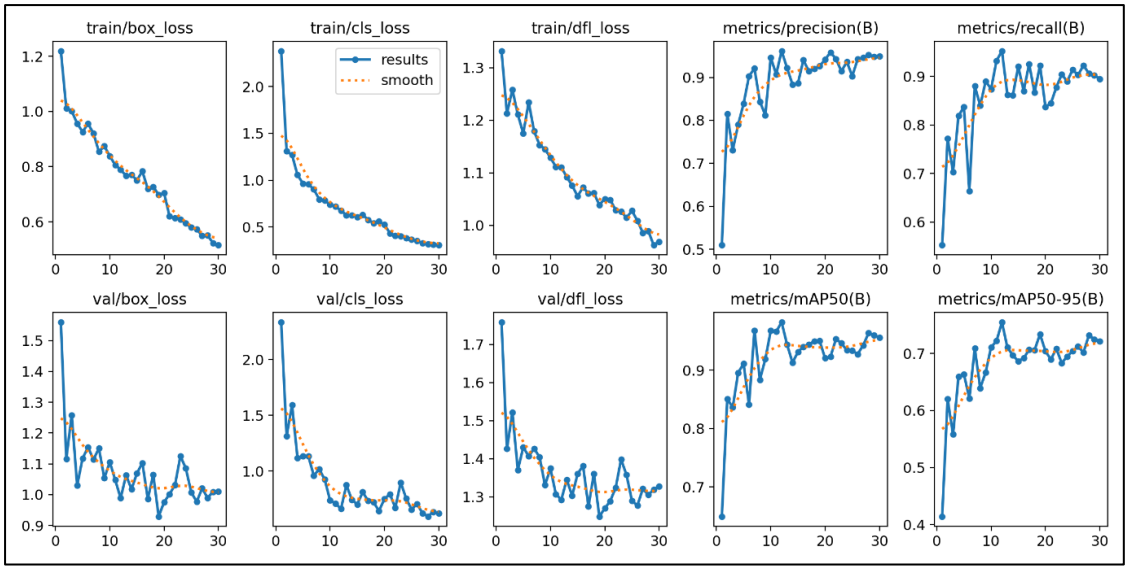


Figure 24: 30 epoch model training results

Validation losses similarly show a general decline, although with more fluctuation, which is typical when evaluating on a smaller or more variable validation set. Importantly, the metrics curves (top right) reveal strong model convergence, with precision and recall both surpassing 0.90 in later epochs, highlighting the model's ability to accurately identify and classify signs without excessive false positives or negatives.

The mean Average Precision at IoU 0.50 (mAP50) reaches approximately 0.95, while the stricter mAP50-95 stabilizes around 0.72, demonstrating the model's robust detection capability across varying intersection-over-union thresholds. These results confirm that the YOLOv8m model not only generalizes well to unseen data but also maintains high localization and classification accuracy, making it suitable for real-time Arabic sign gesture recognition.

The following curves of training and validation in figures 25 and 26 represent the model training of 20 and 40 epochs.

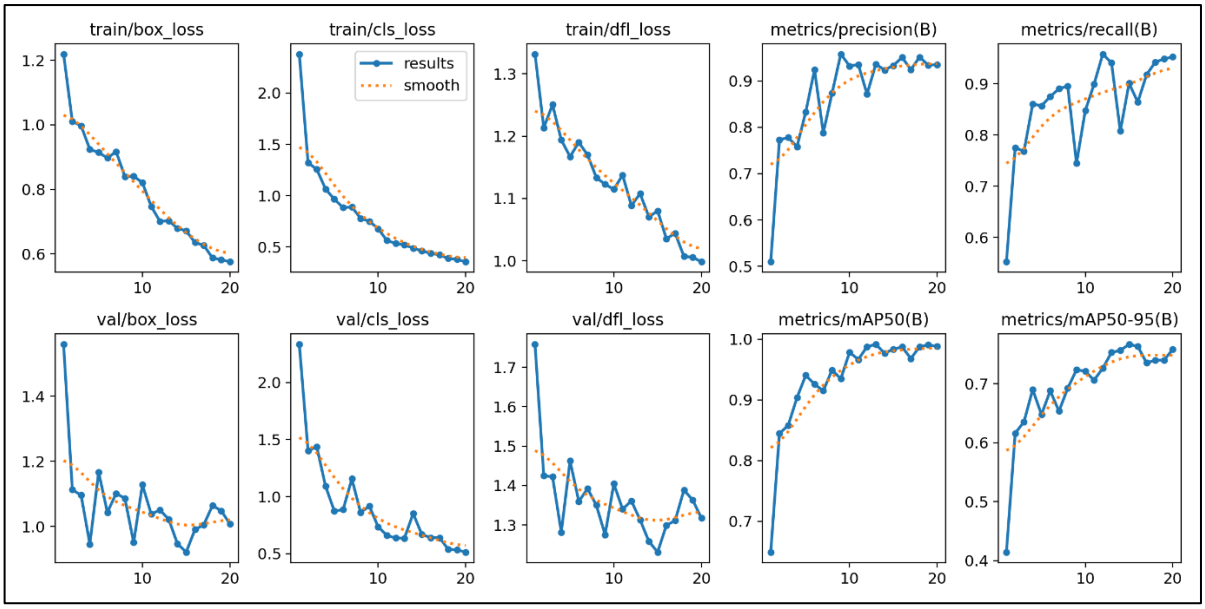


Figure 25: 20 epoch model training results

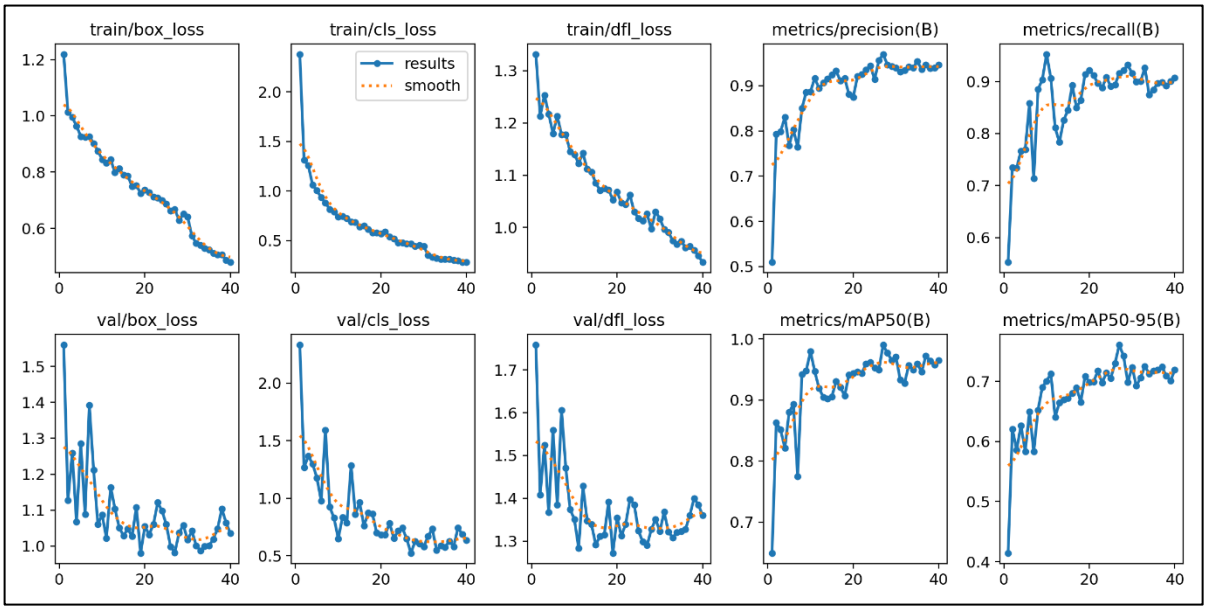


Figure 26: 40 epoch model training results

The normalized confusion matrix in figure 27 reveals strong per-class recognition performance across most of the nine Arabic sign language gestures. Diagonal dominance indicates high true positive rates, with several classes achieving near-perfect classification (normalized values approaching 1.00), such as "شكراً", "أنا", and "ضغط الدم". Off-diagonal entries are minimal, suggesting low confusion between visually similar signs. However, a few signs exhibited minor misclassification, which may be attributed to gesture overlap or intra-class variance, particularly between signs sharing similar hand orientations or motion patterns. The background class was consistently separated from gesture classes, confirming the model's ability to distinguish hand signs from irrelevant frames. These results validate the YOLOv8m model's high discriminative capacity on the custom dataset and its suitability for real-time sign recognition tasks.

To further analyze model performance and learning behavior, visualizations of the initial training and validation batches (Batch 0) are shown in figures 28 and 29 respectively. These figures offer insight into the quality of ground truth annotations, model predictions, and detection consistency during the early stages of training.

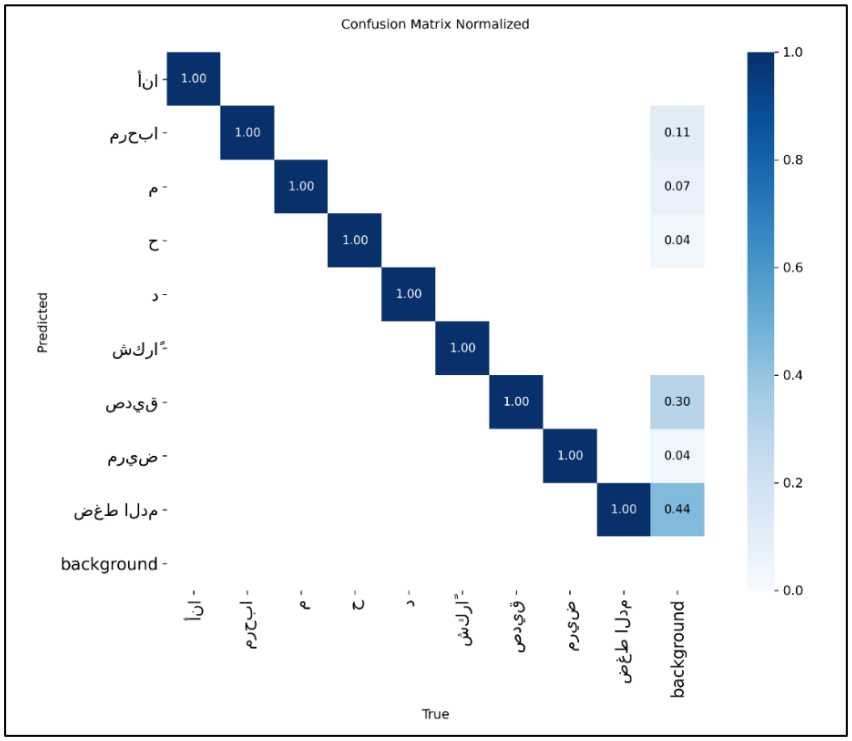


Figure 27: Model's normalized confusion matrix



Figure 28: Model training batch 0

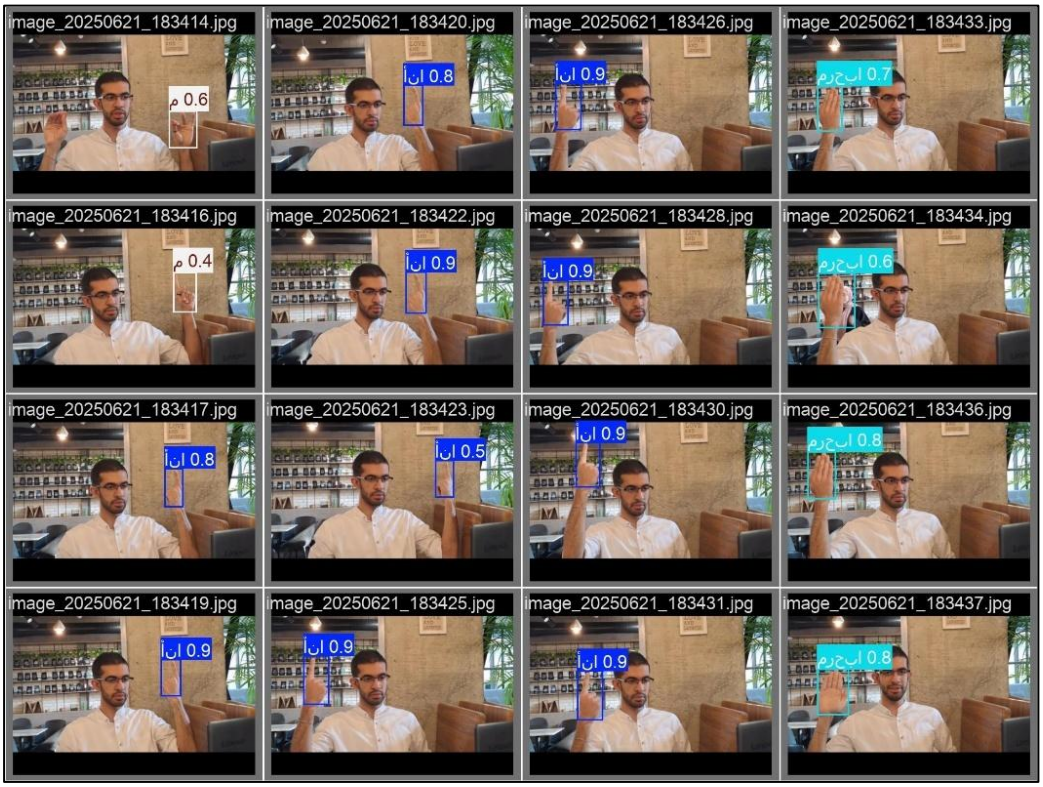


Figure 29: Model validation batch 0

6.5. System integration

The system integrates both hardware and software components into a cohesive real-time image recognition and translation unit built around the Raspberry Pi 5. Functioning as the central processing hub, the Raspberry Pi manages communication between all input, processing, and output elements. Visual data is acquired through Arducam Module V3, which connects to the Pi via a 22-pin ribbon cable attached to the Camera Serial Interface (CSI) port. This setup enables high-speed, low-latency image capture, forming the foundation of the system's visual input pipeline.

Processed outputs are displayed on a 128x64 OLED LCD module interfaced through the Raspberry Pi's GPIO pins using the I2C communication protocol. The OLED module features an integrated I2C interface, allowing communication through just two wires—SDA (data) and SCL (clock)—which reduces GPIO usage while providing a clear and reliable medium for text display. Its higher resolution and flexible layout enable more detailed and readable output.



Figure 30: SignBridge system

The software stack runs on Raspberry Pi OS, a Linux-based operating system that efficiently manages hardware resources and supports the entire software environment. Python serves as the primary programming language, orchestrating all operations and interfacing seamlessly with hardware components and libraries. The PiCamera2 library is employed to access real-time video from the Arducam camera, while OpenCV is used to preprocess frames through tasks such as resizing, color space conversion, and annotation. The following flowchart in figure 31 focuses on the overall system-level integration – hardware and software pipelines.

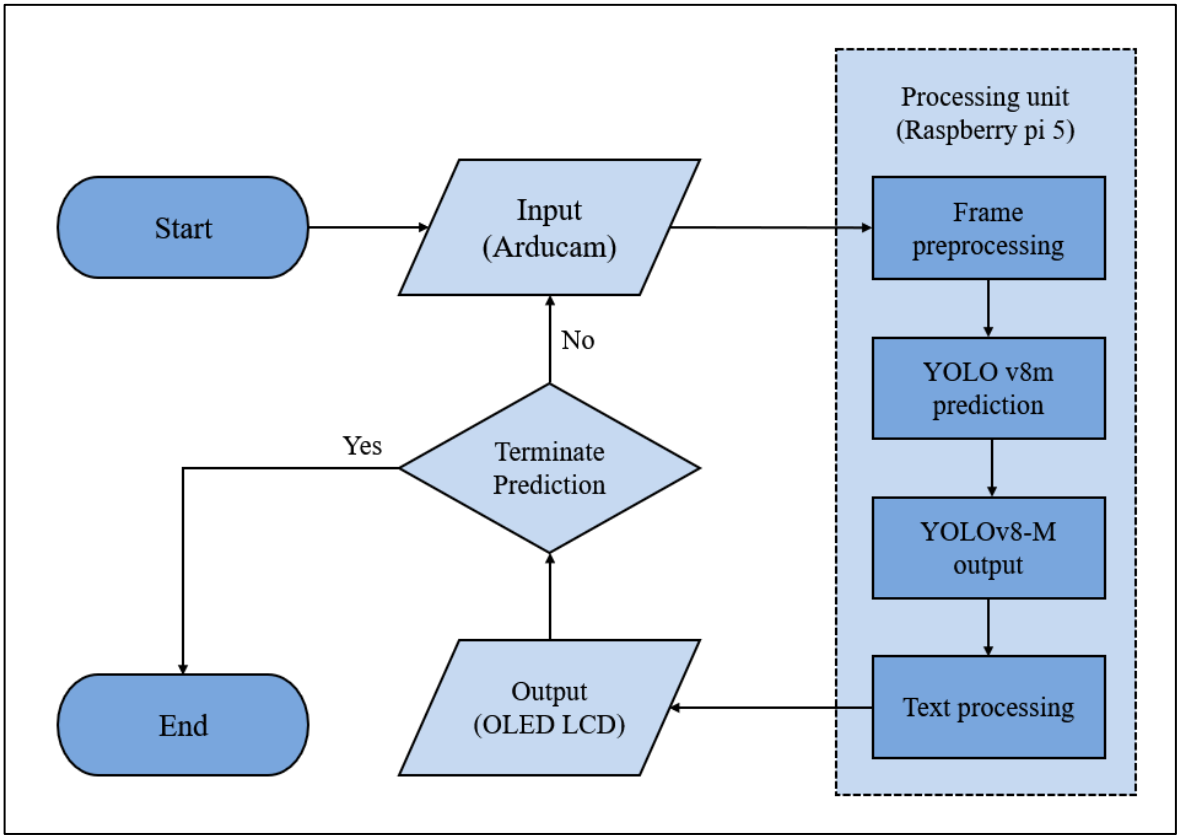


Figure 31: SignBridge system workflow

The following paragraphs and figure 32 describe how the YOLOv8m model handles gesture detection and how the detected outputs are converted into structured Arabic text for user display.

The core image recognition functionality is handled by the Ultralytics YOLO library. The YOLOv8m model performs real-time object detection on video frames captured by the PiCamera2 at a resolution of 640×480 pixels. These frames are resized to 256×256 pixels (imgsz=256) to match the model's processing requirements. A confidence threshold of 0.4 is applied to filter out low-confidence predictions. Once a gesture is detected, the model outputs a bounding box, class label, and confidence score for the recognized hand gesture. These outputs form the basis for interpreting the user's intended sign.

Instead of displaying each prediction directly, the system employs a temporal logic mechanism to stabilize detections and improve output reliability. The algorithm compares successive predictions and accumulates the same label over a time window. If the same label is detected consistently for at least one second, it is accepted as valid. Labels are categorized as either Arabic letters or full words. When a recognized output is a letter, it is appended to a temporary string representing an in-progress word. When a complete word is detected, the current word buffer (if any) is first committed to the sentence before appending the full word.

The sentence is considered complete if no new gesture is detected for three continuous seconds. At that point, the accumulated Arabic letters and words are combined into a grammatically correct sentence. Due to the complexity of Arabic script, the reshaped sentence is processed using the Arabic_reshaper and Bidi libraries to maintain correct right-to-left display order. The final translated sentence is rendered using a custom Arabic font and displayed on the OLED LCD screen.

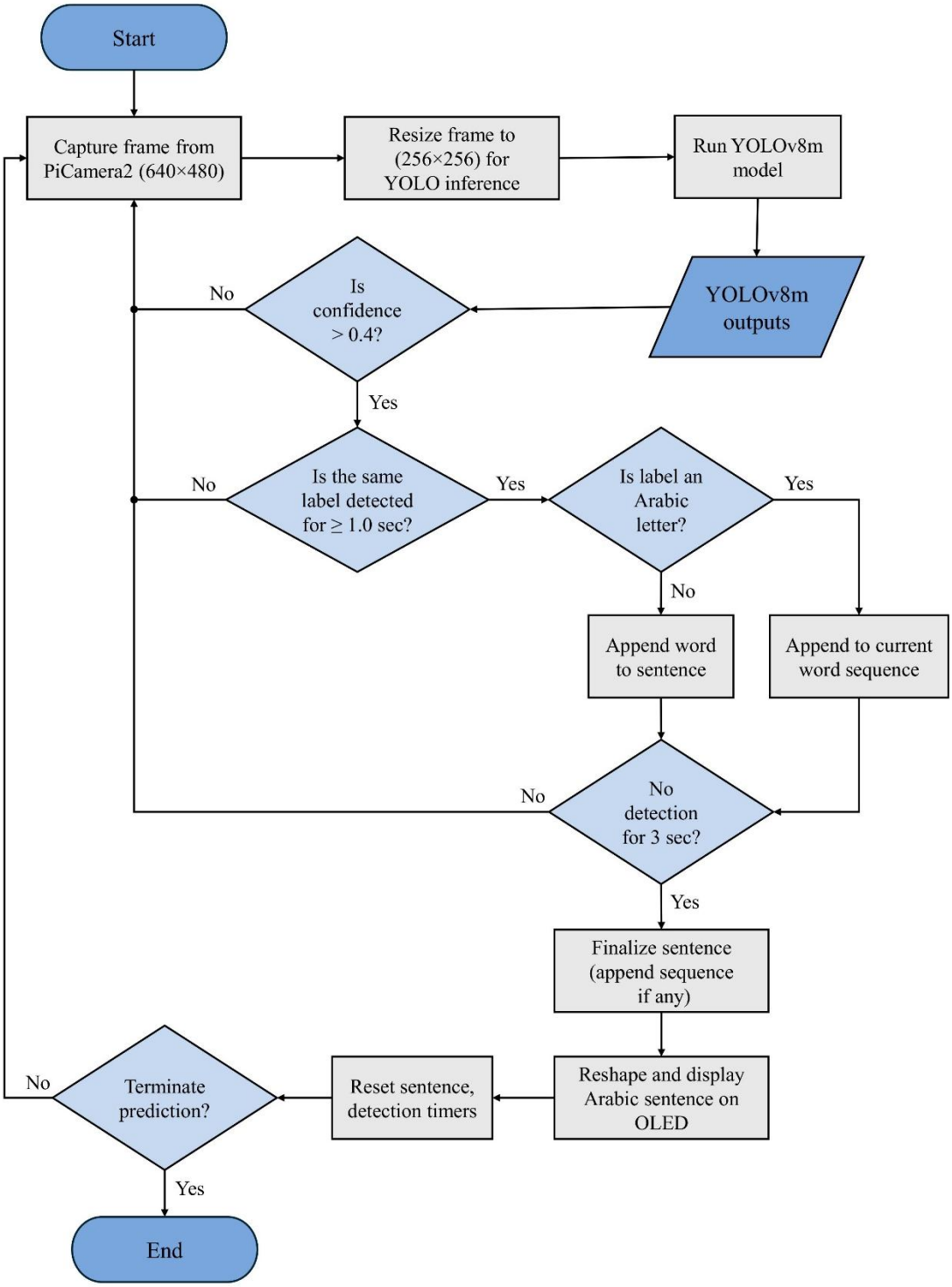


Figure 32: Prediction Flowchart

6.6. Experimentation

The experimental setup consisted of good lighting conditions, with the signer positioned one meter away from the camera, which was placed at a height of one meter. Additionally, the environment had low background noise to minimize interference.

The model's performance was subsequently evaluated on three platforms: The desktop which the model was trained on, a laptop (Lenovo IdeaPad 130-15IKB, Type 81H7, featuring an Intel Core i7-8550U CPU, ST1000LM035-1RK172 HDD, and 8 GB DDR4-2133 MHz RAM), and the Raspberry Pi 5. The following table presents a comparative overview of the results across these devices:

Table 13: YOLOv8m model accuracies over different devices

Device/Sign	ح	د	م	أنا	مرحبا	شكراً	صديق	مريض	ضغط الدم
Desktop	0.87	0.88	0.9	0.85	0.85	0.83	0.83	0.82	0.8
Laptop	0.91	0.81	0.85	0.86	0.85	0.91	0.82	0.7	0.75
Raspberry Pi 5	0.92	0.86	0.93	0.82	0.87	0.75	0.7	0.7	0.74

Table 14: YOLOv8m model performance over different devices

Device/Data	Processing Time	Latency
Desktop	8 – 13 ms	0.2s
Laptop	200 – 350 ms	0.35s
Raspberry Pi 5	400 – 450 ms	1s

6.7. Design limitations

- **Limited Processing Speed**: While the Raspberry Pi is capable of running real-time object detection, its processing power places a cap on the achievable frame rate. This means that, although functional, the system may not match the speed of higher-end hardware when handling rapid or high-resolution input.

- **Model Size Constraints**: The computational and memory limitations of the Raspberry Pi restrict it to running only small to medium-sized YOLO models effectively. Using larger models would exceed its available resources, resulting in slower performance or a potential failure to run.

- **Overheating Risks**: The Raspberry Pi can operate reliably under normal conditions, but when used for sustained processing tasks without proper cooling, it becomes vulnerable to overheating. This may lead to thermal throttling or reduced performance if not addressed with appropriate thermal management solutions.

6.8. Future work

- **Language Model Integration**: Incorporating a Large Language Model (LLM) will allow the system to convert raw sign translations into grammatically correct and contextually coherent sentences. This improvement is intended to enhance the naturalness and clarity of the translated output, making communication more fluid and intuitive for users.

- **Cloud-Based Processing**: Adding support for an online processing mode will enable data to be sent to a centralized server when an internet connection is available. This shift will reduce the computational burden on local hardware, offer access to more advanced processing capabilities, and facilitate easier updates and scalability through cloud infrastructure.

- **Dynamic Gesture Recognition**: Expanding the system's functionality to include dynamic gesture recognition will allow for the interpretation of gestures involving motion and timing, not just static hand shapes. This capability will improve the system's accuracy and range, bringing it closer to real-time, natural sign language communication.

- **Multilanguage Support**: The device will incorporate multilanguage support to accommodate users from diverse linguistic backgrounds. This feature will allow the translated output to be rendered in multiple spoken languages, broadening accessibility and making the system usable across different regions and communities.

6.9. System Schematic Design

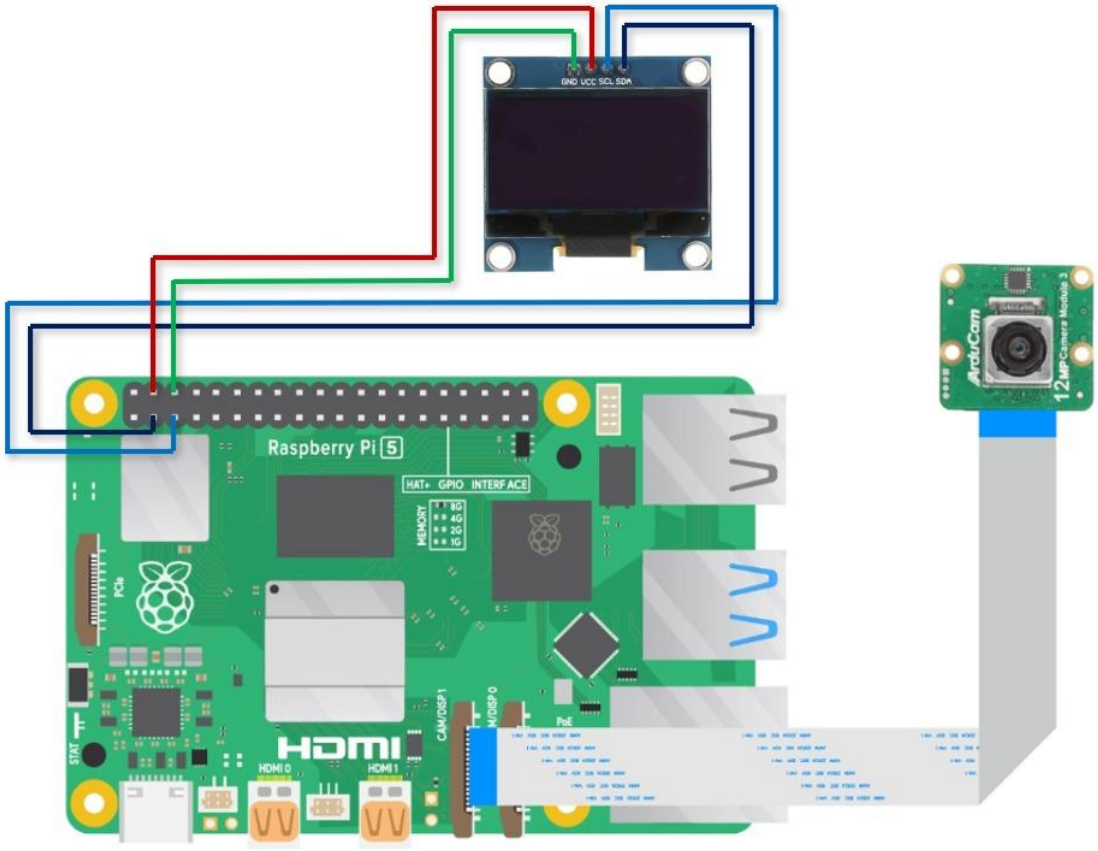


Figure 33: System Schematic Design

7. Project Budget

The budget study was done by collecting information from trusted vendors such as Mikroelectron and Waslleh. The following Table shows the project's estimated costs.

Table 15: Project Budget

Component	Cost
Raspberry Pi 5 (4GB RAM)	85 JD
Arducam Module v3	39 JD
128x64 OLED LCD	6 JD
Power Source	9 JD
Enclosure	12 JD
Stand & Wires	10 JD
Total cost	161 JD

8. Project Schedule

Table 16: Project schedule

Tasks / Weeks	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Identifying project requirements																								
Define the problem statement																								
Highlight project objectives																								
Research & literature survey																								
Identifying required components																								
Outline project design																								
Hardware implementation																								
Software implementation																								
Experimentation & results																								
Final report writing																								

9. Conclusion

SignBridge, a standalone, real-time sign language translation device was successfully developed to bridge communication barriers between the Deaf and hearing communities. The system was built using a Raspberry Pi 5 platform equipped with an autofocus camera and an OLED LCD display, providing an accessible and efficient solution for converting sign language gestures into written text. Advanced image processing, gesture recognition, and machine learning techniques were integrated, with the YOLOv8m object detection model selected for its superior accuracy and performance. Recognition accuracy ranging from 70% to 95% was achieved, and processing latency was maintained at a level suitable for real-time interaction.

The custom SignBridge dataset was created and used to train the model, ensuring it was specifically tailored for Arabic sign language gestures. Extensive testing was conducted with existing public datasets to validate the model's robustness and generalization.

The results demonstrated the potential of technology to enhance social inclusion, educational access, and professional collaboration for individuals with hearing impairments. Future work was proposed to expand the system's capabilities to support multiple languages and bidirectional communication, which would further increase its impact. This project was recognized as a significant step toward fostering equity and improving quality of life through assistive technology innovations.

References

[1] World Health Organization, “Deafness and hearing loss.” Accessed: Jan. 05, 2025. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] O. Macherey and R. P. Carlyon, “Cochlear implants,” Sep. 22, 2014, *Cell Press*. doi: 10.1016/j.cub.2014.06.053.

[3] L. Hassinen and R. Lappalainen, “Acceptance and commitment therapy using finnish sign language: Training counselors in signed ACT for the deaf – A pilot study,” Apr. 01, 2018, *Elsevier Inc.* doi: 10.1016/j.jcbs.2018.02.005.

[4] A. Núñez-Marcos, O. Perez-de-Viñaspre, and G. Labaka, “A survey on Sign Language machine translation,” Mar. 01, 2023, *Elsevier Ltd*. doi: 10.1016/j.eswa.2022.118993.

[5] Alessandro. Duranti, *A Companion to Linguistic Anthropology*. John Wiley & Sons, 2007.

[6] D. M. , G. F. S. and C. D. F. Eberhard, “Languages of the World.” Accessed: Jan. 07, 2025. [Online]. Available: http://www.ethnologue.com.

[7] M. J. Cheok, Z. Omar, and M. H. Jaward, “A review of hand gesture and sign language recognition techniques,” *International Journal of Machine Learning and Cybernetics*, vol. 10, no. 1, pp. 131–153, Jan. 2019, doi: 10.1007/s13042-017-0705-5.

[8] M. Madhiarasan and P. P. Roy, “A Comprehensive Review of Sign Language Recognition: Different Types, Modalities, and Datasets,” Apr. 2022, [Online]. Available: http://arxiv.org/abs/2204.03328

[9] I. Papastratis, C. Chatzikonstantinou, D. Konstantinidis, K. Dimitropoulos, and P. Daras, “Artificial intelligence technologies for sign language,” *Sensors*, vol. 21, no. 17, Sep. 2021, doi: 10.3390/s21175843.

[10] S. Aly and W. Aly, “DeepArSLR: A novel signer-independent deep learning framework for isolated arabic sign language gestures recognition,” *IEEE Access*, vol. 8, pp. 83199–83212, 2020, doi: 10.1109/ACCESS.2020.2990699.

[11] F. S. Botros, A. Phinyomark, and E. J. Scheme, “Electromyography-Based Gesture Recognition: Is It Time to Change Focus from the Forearm to the Wrist?,” *IEEE Trans Industr Inform*, vol. 18, no. 1, pp. 174–184, Jan. 2022, doi: 10.1109/TII.2020.3041618.

[12] T. and A. S. and R. S. and A. S. and K. N. Khan, “Electromyography based Gesture Recognition: An Implementation of Hand Gesture Analysis Using Sensors,” *Sir Syed University Research Journal of Engineering & Technology*, vol. 12, pp. 70–77, Jun. 2022.

[13] M. Hassan, K. Assaleh, and T. Shanableh, “Multiple Proposals for Continuous Arabic Sign Language Recognition,” *Sens Imaging*, vol. 20, no. 1, Dec. 2019, doi: 10.1007/s11220-019-0225-3.

[14] J. Wan, S. Z. Li, Y. Zhao, S. Zhou, I. Guyon, and S. Escalera, “ChaLearn Looking at People RGB-D Isolated and Continuous Datasets for Gesture Recognition.” [Online]. Available: http://www.cbsr.ia.ac.cn/users/jwan/database/isogd.html

[15] A. S. Ghotkar and G. K. Kharate, “Vision based Real Time Hand Gesture Recognition Techniques for Human Computer Interaction,” 2013.

[16] . IEEE Staff, *2010 6th International Colloquium on Signal Processing and Its Applications*. IEEE, 2010.

[17] L. P. Clarke *et al.*, “MRI SEGMENTATION: METHODS AND APPLICATIONS,” 1995.

[18] G. Kumar and P. K. Bhatia, “A detailed review of feature extraction in image processing systems,” in *International Conference on Advanced Computing and Communication Technologies, ACCT*, Institute of Electrical and Electronics Engineers Inc., 2014, pp. 5–12. doi: 10.1109/ACCT.2014.74.

[19] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” 2004.

[20] K. Delac, M. Grgic, and S. Grgic, “Independent comparative study of PCA, ICA, and LDA on the FERET data set,” *Int J Imaging Syst Technol*, vol. 15, no. 5, pp. 252–260, 2005, doi: 10.1002/ima.20059.

[21] M. Fagiani, E. Principi, S. Squartini, and F. Piazza, “Signer independent isolated Italian sign recognition based on hidden Markov models,” *Pattern Analysis and Applications*, vol. 18, no. 2, pp. 385–402, May 2015, doi: 10.1007/s10044-014-0400-z.

[22] R. Cuingnet, M. Chupin, H. Benali, and O. Colliot, “Spatial and anatomical regularization of SVM for brain image analysis.” [Online]. Available: www.adni-info.org

[23] M. Al-Hammadi *et al.*, “Deep learning-based approach for sign language gesture recognition with efficient hand gesture representation,” *IEEE Access*, vol. 8, pp. 192527–192542, 2020, doi: 10.1109/ACCESS.2020.3032140.

[24] D. R. Kothadiya, C. M. Bhatt, T. Saba, A. Rehman, and S. A. Bahaj, “SIGNFORMER: DeepVision Transformer for Sign Language Recognition,” *IEEE Access*, vol. 11, pp. 4730–4739, 2023, doi: 10.1109/ACCESS.2022.3231130.

[25] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, “Convolutional neural networks: an overview and application in radiology,” Aug. 01, 2018, *Springer Verlag*. doi: 10.1007/s13244-018-0639-9.

[26] J. Wu, “Introduction to Convolutional Neural Networks,” 2017.

[27] W. N. Ismail, H. A. Alsalamah, M. M. Hassan, and E. Mohamed, “AUTO-HAR: An adaptive human activity recognition framework using an automated CNN architecture design,” Feb. 01, 2023, *Elsevier Ltd*. doi: 10.1016/j.heliyon.2023.e13636.

[28] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation.” [Online]. Available: http://arxiv.

[29] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” Jan. 2016, [Online]. Available: http://arxiv.org/abs/1506.01497

[30] Z. Lei, S. Z. Li, and S. Zhang, “Occlusion-aware R-CNN: Detecting Pedestrians in a Crowd,” 2018. [Online]. Available: https://www.researchgate.net/publication/326197709

[31] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” Apr. 2018, [Online]. Available: http://arxiv.org/abs/1804.02767

[32] R. Rastgoo, K. Kiani, and S. Escalera, “Sign Language Recognition: A Deep Survey,” Feb. 01, 2021, *Elsevier Ltd*. doi: 10.1016/j.eswa.2020.113794.

[33] D. Klakow and J. Peters, “Testing the correlation of word error rate and perplexity q.” [Online]. Available: www.elsevier.com/locate/specom

[34] R. Rastgoo, K. Kiani, and S. Escalera, “Video-based isolated hand sign language recognition using a deep cascaded model,” *Multimed Tools Appl*, vol. 79, no. 31–32, pp. 22965–22987, Aug. 2020, doi: 10.1007/s11042-020-09048-5.

[35] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, “A Review of Yolo Algorithm Developments,” in *Procedia Computer Science*, Elsevier B.V., 2021, pp. 1066–1073. doi: 10.1016/j.procs.2022.01.135.

[36] M. Bizjak, P. Peer, and Ž. Emeršič, “Mask R-CNN for Ear Detection,” 2019. [Online]. Available: https://github.com/matterport/Mask_RCNN/

[37] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection.” [Online]. Available: http://pjreddie.com/yolo/

[38] S. Cui, Y. Zhou, Y. Wang, and L. Zhai, “Fish Detection Using Deep Learning,” *Applied Computational Intelligence and Soft Computing*, vol. 2020, 2020, doi: 10.1155/2020/3738108.

[39] O. Russakovsky *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” Jan. 2015, [Online]. Available: http://arxiv.org/abs/1409.0575

[40] C. Jiang *et al.*, “Object detection from UAV thermal infrared images and videos using YOLO models,” *International Journal of Applied Earth Observation and Geoinformation*, vol. 112, Aug. 2022, doi: 10.1016/j.jag.2022.102912.

[41] M. Hussain, “YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection,” Jul. 01, 2023, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/machines11070677.

[42] “Home - Ultralytics YOLO Docs.” Accessed: Jun. 28, 2025. [Online]. Available: https://docs.ultralytics.com/#how-can-ultralytics-yolo-be-used-for-real-time-object-tracking

[43] “Model Comparisons: Choose the Best Object Detection Model for Your Project.” Accessed: Jun. 28, 2025. [Online]. Available: https://docs.ultralytics.com/compare/

[44] W. Jia and C. Li, “SLR-YOLO: An improved YOLOv8 network for real-time sign language recognition,” *Journal of Intelligent and Fuzzy Systems*, vol. 46, no. 1, pp. 1663–1680, Jan. 2024, doi: 10.3233/JIFS-235132.

[45] “SignForDeaf.” Accessed: Jan. 05, 2025. [Online]. Available: https://www.signfordeaf.com.

[46] K. Parvez *et al.*, “Measuring effectiveness of mobile application in learning basic mathematical concepts using sign language,” *Sustainability (Switzerland)*, vol. 11, no. 11, Jun. 2019, doi: 10.3390/su11113064.

[47] A. Babour *et al.*, “Intelligent gloves: An IT intervention for deaf-mute people,” *Journal of Intelligent Systems*, vol. 32, no. 1, Jan. 2023, doi: 10.1515/jisys-2022-0076.

[48] Raspberry Pi Ltd, “Raspberry Pi 5 Documentation.” Accessed: Jun. 28, 2025. [Online]. Available: https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

[49] “The Differences Between Raspberry Pi 4 Model B & Raspberry Pi 5 – Kitronik Ltd.” Accessed: Jun. 28, 2025. [Online]. Available:

https://kitronik.co.uk/blogs/resources/the-differences-between-raspberry-pi-4-model-b-raspberry-pi-5

[50] Z. Isherwood and E. Lindo Secco, “A Raspberry Pi computer vision system for self-driving cars.”

[51] M. Abdulhamid, M. Al-Rawi, and O. Odondi, “Computer vision based on raspberry pi system,” *Applied Computer Science*, vol. 16, no. 4, pp. 85–102, 2020, doi: 10.23743/acs-2020-31.

[52] About the Camera Modules, “Raspberry Pi,” 2023, Accessed: Jan. 05, 2025. [Online]. Available: https://www.raspberrypi.com/documentation/accessories/camera.html

[53] “16MP IMX519 - Arducam Wiki.” Accessed: Jun. 28, 2025. [Online]. Available: https://docs.arducam.com/Raspberry-Pi-Camera/Native-camera/16MP-IMX519/

[54] Raspberry Pi Ltd, “Raspberry Pi Case for Raspberry Pi 5.” Accessed: Jun. 28, 2025. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5-case/

[55] S. Raschka, J. Patterson, and C. Nolet, “Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence,” Apr. 01, 2020, *MDPI AG*. doi: 10.3390/info11040193.

[56] T. Toe and T. Z. Rong, “Artificial Intelligence with Python.” [Online]. Available: https://link.springer.com/bookseries/16715

[57] A. A. I. Sidig, H. Luqman, S. Mahmoud, and M. Mohandes, “KArSL: Arabic Sign Language Database,” *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 20, no. 1, pp. 1–19, Apr. 2021, doi: 10.1145/3423420.

[58] “Model Comparison: YOLOv9 vs. YOLOv8 for Object Detection.” Accessed: Jun. 29, 2025. [Online]. Available: https://docs.ultralytics.com/compare/yolov9-vs-yolov8/#weaknesses

[59] “YOLOv8 Architecture; Deep Dive into its Architecture -Yolov8.” Accessed: Jun. 29, 2025. [Online]. Available: https://yolov8.org/yolov8-architecture/

[60] “YOLOv8 Architecture; Deep Dive into its Architecture -Yolov8.” Accessed: Jun. 29, 2025. [Online]. Available: https://yolov8.org/yolov8-architecture/

[61] “Explore Ultralytics YOLOv8 - Ultralytics YOLO Docs.” Accessed: Jun. 29, 2025. [Online]. Available: https://docs.ultralytics.com/models/yolov8/#performance-metrics

[62] J. Banarase, M. Gawande, S. Kale, and S. Mahalle, “Basketball Virtual Referee,” *International Research Journal of Engineering and Technology*, 2024, [Online]. Available: www.irjet.net